

Minecraft Forge 1.20.1 文档 (中文翻译)

MinecraftForge

2026-06-22

Contents

MinecraftForge 文档	4
贡献本文档	5
风格指南	5
Forge 入门指南	5
先决条件	5
从零开始模组开发	5
自定义模组信息	6
构建和测试你的模组	7
模组文件	7
mods.toml	7
模组入口点	10
组织你的模组	10
包结构	10
类命名规范	11
从多种方法中选择一种	11
版本控制	11
示例	12
注册表	12
注册方法	13
引用已注册对象	14
创建自定义 Forge 注册表	15
处理缺失条目	15
Minecraft 中的端	16
不同类型的端	16
执行端特定操作	16
常见错误	17
编写单端模组	17
事件	18
创建事件处理器	18
取消事件	19
结果	19
优先级	19
子事件	19
模组事件总线	20

模组生命周期	20
注册事件	20
数据生成	21
通用设置	21
端侧设置	21
模组间通信 (InterModComms)	21
资源	21
ResourceLocation	21
国际化与本地化	21
语言文件	22
在方块和物品中的使用	22
本地化方法	22
方块	23
创建方块	23
注册方块	23
延伸阅读	23
方块状态	23
遗留行为	23
状态的引入	24
方块状态的正确使用	24
实现方块状态	24
使用 BlockState	25
方块实体	25
注册	25
创建 BlockEntity	25
在 BlockEntity 中存储数据	26
使 BlockEntity 进行 tick	26
将数据同步到客户端	26
方块实体渲染器	27
创建 BER	27
注册 BER	27
物品	28
创建物品	28
创造模式标签页	28
注册物品	29
BlockEntityWithoutLevelRenderer	29
使用 BlockEntityWithoutLevelRenderer	29
网络	29
SimpleImpl	29
入门	30
版本检查器	30
注册数据包	30
处理数据包	30
发送数据包	31
实体	31
生成数据	31
动态数据	32

粒子	32
声音	33
能力系统	33
持久化数据 (SavedData)	34
编解码器 (Codec)	34
菜单	35
MenuType	35
AbstractContainerMenu	36
打开菜单	38
屏幕	39
相对坐标	39
GuiGraphics	40
Renderable	40
GuiEventListener	41
NarratableEntry	41
屏幕子类型	41
AbstractContainerScreen	42
注册 AbstractContainerScreen	44
根变换	44
渲染类型	44
原版值	45
自定义值	45
部件可见性	45
面数据	45
Elements 模型	45
生成的物品模型	46
参数	46
自定义模型加载器	47
BakedModel	47
变换	47
ItemOverrides	48
资源包	48
模型	48
纹理着色	48
物品属性	49
数据包	49
配方	49
自定义配方	49

原料 (Ingredient)	50
非数据包配方	50
战利品表	50
全局战利品修改器	51
标签	51
进度	52
条件加载数据	52
数据生成器	53
现有文件	53
生成器模式	53
数据提供者 (Data Providers)	53
模型生成	54
语言文件生成	54
声音定义生成	55
配方生成	55
战利品表生成	56
标签生成	56
进度生成	57
全局战利品修改器生成	57
数据包注册表对象生成	57
配置	58
按键映射	58
游戏测试	59
Forge 更新检查器	59
调试分析器	60
访问转换器	60
开始 Forge 开发	61
Pull Request 指南	61
旧版本文档	62
移植到 Minecraft 1.20	62

MinecraftForge 文档

这是 MinecraftForge (Minecraft 模组开发 API) 的官方文档。

本文档仅针对 Forge，这不是 Java 教程。

如果你想为文档做出贡献，请阅读贡献文档。

贡献本文档

你可以通过 GitHub 上的 PR 进行贡献。

本文档旨在提供说明性内容。请解释如何操作，并将其拆分为合理的模块。我们另有 Wiki 站点可以收录更全面的代码示例。

我们的受众是所有想要了解如何使用 Forge 构建模組的开发人员。

请不要试图将本文档变成 Java 开发教程——它的目标读者是已经了解 Java 类以及其他 Java 基础结构的开发人员。

风格指南

!!! important 请使用**两个空格**进行缩进，而不是制表符。

标题应使用标准的标题格式进行大写。例如：

- Guide For Contributing to This Documentation
- Building and Testing Your Mod

基本上，除了非重要的单词外，所有单词都应大写。

拼写、语法和句法应遵循美式英语规范。此外，建议使用分立的单词而非缩略形式（例如使用“are not”而不是“aren't”）。

请使用等号和短划线作为标题下划线，而不是 # 和 ##。对于 h3 及更低级别，可以使用 ### 等标记。本文件的源代码包含了等号和短划线标题下划线的示例。等号下划线表示 h1 标题，短划线下划线表示 h2 标题。

在代码片段之外引用字段和方法时，应使用 # 分隔符（例如 ClassName#methodName）。内部类应使用 \$ 分隔符（例如 ClassName\$InnerClassName）。

JSON 代码片段应使用 js 语法高亮。

所有链接的位置应在页面底部指定。任何内部链接应通过相对路径引用页面。

警告块（以 !!! <type> 表示）必须按照文档中的格式编写；否则可能导致渲染错误。

Forge 入门指南

如果你从未制作过 Forge 模组，本节将提供搭建 Forge 开发环境所需的最基本信息。本文档的其他部分将介绍后续的学习方向。

先决条件

- 安装 Java 17 开发工具包 (JDK) 和 64 位 Java 虚拟机 (JVM)。Forge 推荐并官方支持 Eclipse Temurin。

!!! warning 确保你使用的是 64 位 JVM。验证方法之一是在终端中运行 `java -version`。使用 32 位 JVM 会在使用 ForgeGradle 时导致一些问题。

- 熟悉集成开发环境 (IDE)。
 - 建议使用带有 Gradle 集成的 IDE。

从零开始模组开发

1. 从 Forge 文件站点 下载模组开发工具包 (MDK)：点击 ‘Mdk’，等待一段时间后点击右上角的 ‘Skip’ 按钮。建议尽可能下载最新版本的 Forge。
2. 将下载的 MDK 解压到一个空目录中。这将作为你的模组目录，其中应包含一些 Gradle 文件和一个包含示例模组的 src 子目录。

!!! note 以下一些文件可以在不同模组之间重复使用：

- * ``gradle`` 子目录
- * ``build.gradle``
- * ``gradlew``
- * ``gradlew.bat``
- * ``settings.gradle``

``src`` 子目录不需要在工作区之间复制；不过，如果之后创建了 `java`(`src/main/java`)` 和资源 (``src/main/resources``) 目录，可能需要刷新 Gradle 项目。

3. 打开你选择的 IDE:

- Forge 仅明确支持在 Eclipse 和 IntelliJ IDEA 上进行开发，但 Visual Studio Code 也有额外的运行配置。尽管如此，从 Apache NetBeans 到 Vim/Emacs 的任何环境都可以使用。
- Eclipse 和 IntelliJ IDEA 的 Gradle 集成（默认已安装并启用）将在导入或打开时处理其余的工作区初始设置。这包括从 Mojang、MinecraftForge 等处下载必要的包。Visual Studio Code 需要 ‘Gradle for Java’ 插件才能实现相同的功能。
- 对于几乎所有对其关联文件（例如 `build.gradle`、`settings.gradle` 等）的更改，都需要调用 Gradle 来重新评估项目。某些 IDE 带有 ‘Refresh’ 按钮来实现此操作；也可以通过终端使用 `gradlew` 完成。

4. 为你选择的 IDE 生成运行配置:

- **Eclipse:** 运行 `genEclipseRuns` 任务。
- **IntelliJ IDEA:** 运行 `genIntelliJRuns` 任务。如果出现 “module not specified” 错误，请将 `ideaModule` 属性设置为主模块（通常为 `${project.name}.main`）。
- **Visual Studio Code:** 运行 `genVSCodeRuns` 任务。
- **其他 IDE:** 你可以直接使用 `gradle run*` 运行配置（例如 `runClient`、`runServer`、`runData`、`runGameTestServer`）。这些也可以与支持的 IDE 一起使用。

自定义模组信息

编辑 `build.gradle` 文件以自定义模组的构建方式（例如文件名、工件版本等）。

!!! important 除非你知道自己在做什么，否则**不要**编辑 `settings.gradle`。该文件指定了 ForgeGradle 上传到的仓库。

推荐的 build.gradle 自定义设置

模组 ID 替换 将所有出现的 `examplemod` (包括 `mods.toml` 和主模组文件) 替换为你的模组 ID。这还包括通过设置 `base.archivesName` 更改构建文件的名称（通常设置为你的模组 ID）。

```
// 在 build.gradle 中
base.archivesName = 'mymod'
```

组 ID `group` 属性应设置为你自己的顶级包，它应该是你拥有的域名或你的电子邮件地址：

类型	值	顶级包
域名	example.com	com.example
子域名	example.github.io	io.github.example
电子邮件	example@gmail.com	com.gmail.example

```
// 在 build.gradle 中
group = 'com.example'
```

Java 源代码 (`src/main/java`) 中的包现在也应遵循此结构，并使用一个内部包表示模组 ID：

```
com
- example (在 group 属性中指定的顶级包)
  - mymod (模组 ID)
    - MyMod.java (重命名后的 ExampleMod.java)
```

版本 将 `version` 属性设置为当前模组版本。我们建议使用 Maven 版本控制的一种变体。

```
// 在 build.gradle 中
version = '1.20-1.0.0.0'
```

额外配置

其他配置可以在 ForgeGradle 文档中找到。

构建和测试你的模组

1. 要构建模组，运行 `gradlew build`。默认情况下，这将在 `build/libs` 中输出一个名为 `[archivesBaseName]-[version].jar` 的文件。此文件可以放入已安装 Forge 的 Minecraft 设置的 `mods` 文件夹中进行分发。
2. 要在测试环境中运行模组，你可以使用生成的运行配置或使用关联的任务（例如 `gradlew runClient`）。这将从运行目录（默认为 `'run'`）启动 Minecraft，并附带指定的源集。默认 MDK 包含 `main` 源集，因此在 `src/main/java` 中编写的任何代码都将生效。
3. 如果你正在运行专用服务器，无论是通过运行配置还是 `gradlew runServer`，服务器最初都会立即关闭。你需要通过编辑运行目录中的 `eula.txt` 文件来接受 Minecraft EULA。接受后，服务器将加载，然后可以通过直接连接到 `localhost` 进行访问。

!!! note 你应该始终在专用服务器环境中测试你的模组。这包括仅客户端的模组，因为它们在服务器上加载时不应执行任何操作。

模组文件

模组文件负责确定哪些模组被打包到你的 JAR 中、在 `'Mods'` 菜单中显示什么信息以及你的模组应如何在游戏中加载。

mods.toml

`mods.toml` 文件定义了你的模组的元数据。它还包含在 `'Mods'` 菜单中显示的其他信息以及你的模组应如何加载到游戏中。

该文件使用 Tom's Obvious Minimal Language，即 TOML 格式。该文件必须存放在所用源集的资源目录下的 `META-INF` 文件夹中（对于 `main` 源集为 `src/main/resources/META-INF/mods.toml`）。一个 `mods.toml` 文件可能如下所示：

```
modLoader="javafml"
loaderVersion="[46,)"

license="All Rights Reserved"
issueTrackerURL="https://github.com/MinecraftForge/MinecraftForge/issues"
showAsResourcePack=false

[[mods]]
  modId="examplemod"
  version="1.0.0.0"
  displayName="Example Mod"
  updateJSONURL="https://files.minecraftforge.net/net/minecraftforge/forge/promotions_slim.json"
  displayURL="https://minecraftforge.net"
  logoFile="logo.png"
  credits=" 我要感谢我的母亲和父亲。"
  authors=" 作者"
  description='''
  让你可以将泥土合成钻石。这是一个存在了千万年的传统模组。它是远古的。神圣的 Notch 创造了它。Jeb 将其彩虹化。Dinnerbone 把它颠倒了。等等。
  '''
  displayTest="MATCH_VERSION"

[[dependencies.examplemod]]
  modId="forge"
  mandatory=true
  versionRange="[46,)"
  ordering="NONE"
  side="BOTH"

[[dependencies.examplemod]]
  modId="minecraft"
  mandatory=true
  versionRange="[1.20]"
  ordering="NONE"
  side="BOTH"
```

mods.toml 分为三个部分：非模组特定属性（与模组文件关联）、模组属性（每个模组一个部分）和依赖配置（每个模组或模组组的依赖一个部分）。下面将解释与 mods.toml 文件相关的每个属性，其中 required 表示必须指定一个值，否则将抛出异常。

非模组特定属性

非模组特定属性是与 JAR 本身关联的属性，指示如何加载模组以及任何额外的全局元数据。

属性	类型	默认值	描述	示例
modLoader	string	必填	模组使用的语言加载器。可用于支持替代语言结构，例如 Kotlin 对象作为主文件，或不同的入口点确定方法，例如接口或方法。Forge 提供了 Java 加载器 "javafml" 和低代码/无代码加载器 "lowcodefml"。	"javafml"
loaderVersion	string	必填	语言加载器的可接受版本范围，以 Maven 版本范围表示。对于 javafml 和 lowcodefml，版本是 Forge 版本的主版本号。	"[46,)"
license	string	必填	此 JAR 中模组所使用的许可证。建议将其设置为你使用的 SPDX 标识符 和/或许可证链接。你可以访问 https://choosealicense.com/ 来帮助选择你想要使用的许可证。	"MIT"
showAsResourcePack	boolean	false	当为 true 时，模组的资源将作为单独的资源包显示在 'Resource Packs' 菜单中，而不是与 'Mod resources' 包合并。	true
services	array	[]	你的模组使用的服务数组。这是作为 Forge 实现的 Java 平台模块系统中为模组创建的模块的一部分而被消费的。	["net.minecraftforge.forgespi.langu
properties	table	{}	一个替换属性表。由 StringSubstitutor 用于将 \${file.<key>} 替换为其对应的值。目前仅用于替换模组特定属性中的 version。	{ "example" = "1.2.3" } 通过 \${file.example} 引用
issueTrackerURL	string	无	代表报告和跟踪模组问题位置的 URL。	"https://forums.minecraftforge.net/"

!!! important services 属性在功能上等同于在模块中指定 uses 指令，允许加载给定类型的服务。

模组特定属性

模组特定属性使用 [[mods]] 标题绑定到指定的模组。这是一个表格数组；所有键/值属性将附加到该模组，直到下一个标题。

```
# examplemod1 的属性
[[mods]]
modId = "examplemod1"

# examplemod2 的属性
[[mods]]
modId = "examplemod2"
```

属性	类型	默认值	描述	示例
modId	string	必填	表示此模组的唯一标识符。ID 必须匹配 <code>^[a-z][a-z0-9_]{1,63}\$</code> (长度为 2-64 个字符；以小写字母开头；由小写字母、数字或下划线组成)。	"examplemod"

属性	类型	默认值	描述	示例
namespace	string	modId 的值	模组的覆盖命名空间。命名空间必须匹配 <code>^[a-z][a-z0-9_-]{1,63}\$</code> (长度为 2-64 个字符；以小写字母开头；由小写字母、数字、下划线、点或短划线组成)。目前未使用。	"example"
version	string	"1"	模组的版本，最好使用 Maven 版本控制的变体。当设置为 <code>\$ {file.jarVersion}</code> 时，将替换为 JAR 清单中 Implementation-Version 属性的值（在开发环境中显示为 <code>0.0NONE</code> ）。	"1.20-1.0.0.0"
displayName	string	modId 的值	模组的友好名称、用于在屏幕上表示模组（例如模组列表、模组不匹配）。	" 示例模组"
description	string	"MISSING DESCRIPTION"	模组列表中显示的模组描述。建议使用多行字面量字符串。	" 这是一个示例。"
logoFile	string	无	用于模组列表屏幕的图像文件的名称和扩展名。徽标必须位于 JAR 的根目录或源集的根目录中（例如 main 源集的 <code>src/main/resources</code> ）。	"example_logo.png"
logoBlur	boolean	true	是否使用 <code>GL_LINEAR*</code> (true) 或 <code>GL_NEAREST*</code> (false) 来渲染 logoFile。	false
updateJSONURL	string	无	一个指向 JSON 文件的 URL，由更新检查器用于确保你正在玩的模组是最新版本。	"https://files.minecraftforge.net/net/minecraftforge/forge/promotions_slim.json"
features	table	{}	见 'features'。	{ java_version = "17" }
modproperties	table	{}	与此模组关联的键/值表。目前未被 Forge 使用，主要用于模组自身使用。	{ example = "value" }
modUrl	string	无	指向模组下载页面的 URL。目前未使用。	"https://files.minecraftforge.net/"
credits	string	无	在模组列表屏幕上显示的模组鸣谢和致谢。	" 那位在那里的某人。"
authors	string	无	在模组列表屏幕上显示的模组作者。	" 示例人物"
displayURL	string	无	在模组列表屏幕上显示的模组展示页面 URL。	"https://minecraftforge.net/"
displayTest	string	"MATCH_VERSION"	见 'sides'。	"NONE"

特性 (Features) 特性系统允许模组在加载系统时要求某些设置、软件或硬件可用。当某个特性未满足时，模组加载将失败，并通知用户该需求。目前，Forge 提供以下特性：

特性	描述	示例
java_version	Java 版本的可接受版本范围，以 Maven 版本范围 表示。这应为 Minecraft 所使用的受支持版本。	"[17,)"

依赖配置

模组可以指定其依赖关系，Forge 在加载模组之前会检查这些依赖关系。这些配置使用表格数组 `[[dependencies.<modid>]]` 创建，其中 modid 是依赖所属模组的标识符。

属性	类型	默认值	描述	示例
modId	string	必填	作为依赖添加的模组的标识符。	"example_library"
mandatory	boolean	必填	当此依赖未满足时游戏是否应崩溃。	true
versionRange	string	" "	语言加载器的可接受版本范围，以 Maven 版本范围 表示。空字符串匹配任何版本。	"[1, 2)"

属性	类型	默认值	描述	示例
ordering	string	"NONE"	定义模组必须在此依赖之前 ("BEFORE") 还是之后 ("AFTER") 加载。如果排序不重要, 返回 "NONE"。	"AFTER"
side	string	"BOTH"	依赖必须存在的物理端: "CLIENT"、"SERVER" 或 "BOTH"。	"CLIENT"
referralUrl	string	无	指向依赖下载页面的 URL。目前未使用。	"https://library.example.com/"

!!! warning 两个模组的 ordering 可能会导致因循环依赖而崩溃: 例如, 模组 A 必须在模组 B "BEFORE" 加载, 而模组 B 又必须在模组 A "BEFORE" 加载。

模组入口点

现在 mods.toml 已填写完成, 我们需要提供一个入口点来开始编写模组程序。入口点本质上是执行模组的起点。入口点本身由 mods.toml 中使用的语言加载器决定。

javafml 和 @Mod

javafml 是 Forge 为 Java 编程语言提供的语言加载器。入口点使用带有 @Mod 注解的公共类定义。@Mod 的值必须包含 mods.toml 中指定的模组 ID 之一。然后, 所有初始化逻辑 (例如注册事件、添加 DeferredRegister) 都可以在类的构造方法中指定。模组总线可以从 FMLJavaModLoadingContext 获取。

```
@Mod("examplemod") // 必须与 mods.toml 匹配
public class Example {

    public Example() {
        // 在此处编写初始化逻辑
        var modBus = FMLJavaModLoadingContext.get().getModEventBus();

        // ...
    }
}
```

lowcodefml

lowcodefml 是一种语言加载器, 用于将数据包和资源包作为模组分发, 而无需代码入口点。它指定为 lowcodefml 而非 nocodefml, 是为了将来可能需要进行少量编码的小幅扩展。

组织你的模组

结构良好的模组有利于维护、贡献代码以及更清晰地理解底层代码库。下面列出了一些来自 Java、Minecraft 和 Forge 的建议。

!!! note 你不必遵循以下建议; 你可以按任何你认为合适的方式组织模组。不过, 我们仍然强烈建议这样做。

包结构

在组织模组时, 选择一个唯一的顶级包结构。许多程序员会对不同的类、接口等使用相同的名称。Java 允许类只要位于不同的包中就可以具有相同的名称。因此, 如果两个类具有相同的包和相同的名称, 那么只会加载其中一个, 这很可能会导致游戏崩溃。

```
a.jar
- com.example.ExampleClass
b.jar
- com.example.ExampleClass // 此类通常不会被加载
```

这在涉及模块加载时甚至更为重要。如果在不同模块中存在两个相同名称的包下的类文件, 这将导致模组加载器在启动时崩溃, 因为模组模块被导出到游戏和其他模组。

```

module A
- package X
- class I
- class J
module B
- package X // 此包将导致模组加载器崩溃，因为已存在一个导出包 X 的模块
- class R
- class S
- class T

```

因此，你的顶级包应该是一些你所拥有的东西：域名、电子邮件地址、你网站的子域名等。甚至可以是你名字或用户名，只要你能保证它在预期目标中是唯一可识别的。

类型	值	顶级包
域名	example.com	com.example
子域名	example.github.io	io.github.example
电子邮件	example@gmail.com	com.gmail.example

下一级包应该是你的模组 ID（例如 com.example.examplemod，其中 examplemod 是模组 ID）。这将保证，除非你有两个具有相同 ID 的模组（这种情况不应发生），你的包不会出现加载问题。

你可以在 Oracle 的教程页面上找到一些额外的命名规范。

子包组织

除了顶级包之外，强烈建议将模组的类分散到子包中。有两种主要的方法：

- **按功能分组**：为具有共同用途的类创建子包。例如，方块可以放在 block 或 blocks 下，实体放在 entity 或 entities 下等。Mojang 使用此结构，并使用单词的单数形式。
- **按逻辑分组**：为具有共同逻辑的类创建子包。例如，如果你正在创建一种新型的工作台，你会将其方块、菜单、物品等放在 feature.crafting_table 下。

客户端、服务器和数据包 通常，仅适用于特定端或运行时的代码应与其它类隔离在单独的子包中。例如，与数据生成相关的代码应放在 data 包中，而仅适用于专用服务器的代码应放在 server 包中。

然而，强烈建议将仅客户端代码隔离在 client 子包中。这是因为专用服务器无法访问 Minecraft 中任何仅客户端的包。因此，使用专门的包可以提供一个好的合理性检查，以验证你没有在模组中跨端访问。

类命名规范

统一的类命名规范有助于解读类的用途或轻松定位特定类。

类通常以其类型作为后缀，例如：

- 一个名为 PowerRing 的 Item -> PowerRingItem。
- 一个名为 NotDirt 的 Block -> NotDirtBlock。
- 一个烤箱的菜单 -> OvenMenu。

!!! note Mojang 通常对所有类（实体除外）遵循类似的结构。实体仅由其名称表示（例如 Pig、Zombie 等）。

从多种方法中选择一种

有许多方法可以执行特定任务：注册对象、监听事件等。通常建议保持一致，使用单一的方法来完成特定任务。这不仅可以改善代码格式，还可以避免可能发生的任何奇怪的交互或冗余（例如事件监听器执行两次）。

版本控制

在一般项目中，通常使用语义化版本控制（格式为 MAJOR.MINOR.PATCH）。然而，在模组开发中，使用格式 MCVERSION-MAJORMOD.MAJORAPI.MINOR.PATCH 可能更为有利，以便能够区分模组的破坏性世界更新和破坏性 API 变更。

!!! important Forge 使用 Maven 版本范围来比较版本字符串，这与语义化版本控制 2.0.0 规范并非完全兼容，例如“预发布”标签。

示例

以下是可以递增各个变量的示例列表。

- MCVERSION
 - 始终与模组所对应的 Minecraft 版本匹配。
- MAJORMOD
 - 移除物品、方块、方块实体等。
 - 更改或移除先前存在的机制。
 - 升级到新的 Minecraft 版本。
- MAJORAPI
 - 更改枚举的变量顺序或变量本身。
 - 更改方法的返回类型。
 - 完全移除公开方法。
- MINOR
 - 添加物品、方块、方块实体等。
 - 添加新机制。
 - 弃用公开方法。(这不属于 MAJORAPI 递增, 因为它不会破坏 API。)
- PATCH
 - Bug 修复。

当递增任何变量时, 所有较小的变量应重置为 0。例如, 如果 MINOR 递增, 则 PATCH 应变为 0。如果 MAJORMOD 递增, 则所有其他变量应变为 0。

进行中的工作

如果你处于模组的初始开发阶段 (在任何正式发布之前), MAJORMOD 和 MAJORAPI 应始终为 0。每次构建模组时, 只应更新 MINOR 和 PATCH。一旦你构建了正式版本 (大多数情况下具有稳定的 API), 应将 MAJORMOD 递增到版本 1.0.0.0。对于任何进一步的开发阶段, 请参考本文档的预发布和发布候选部分。

多个 Minecraft 版本

如果模组升级到新版本的 Minecraft, 并且旧版本仅会收到错误修复, 则 PATCH 变量应根据升级前的版本进行更新。如果模组在旧版本和新版本的 Minecraft 中都仍在积极开发中, 建议将版本附加到**两个**构建号上。例如, 如果由于 Minecraft 版本变更, 模组升级到版本 3.0.0.0, 则旧模组也应更新到 3.0.0.0。例如, 旧版本将变为 1.7.10-3.0.0.0, 而新版本将变为 1.8-3.0.0.0。如果在为较新的 Minecraft 版本构建时没有任何更改, 则除了 Minecraft 版本之外的所有变量都应保持不变。

最终版本

当停止支持某个 Minecraft 版本时, 该版本的最后一个构建应添加 -final 后缀。这表示该模组将不再支持指定的 MCVERSION, 玩家应升级到更新版本的模组以继续接收更新和错误修复。

预发布

也可以预发布正在进行中的功能, 这意味着发布尚未完全完成的新功能。这可以看作是某种“beta”版本。这些版本应附加 -betaX, 其中 X 是预发布的编号。(本指南不使用 -pre, 因为在撰写本文时, 它不是 -beta 的有效别名。)请注意, 已发布的版本和初始版本之前的版本不能进入预发布; 应在添加 -beta 后缀之前相应地更新变量 (主要是 MINOR, 但 MAJORAPI 和 MAJORMOD 也可以进行预发布)。初始版本之前的版本仅仅是进行中的工作构建。

发布候选

发布候选作为实际版本变更之前的预发布版本。这些版本应附加 -rcX, 其中 X 是发布候选的编号, 理论上只应为错误修复而递增。已发布的版本不能有发布候选; 应在添加 -rc 后缀之前相应地更新变量 (主要是 MINOR, 但 MAJORAPI 和 MAJORMOD 也可以进行预发布)。当发布候选版本作为稳定版本发布时, 它可以与最后一个发布候选完全一样, 或者包含更多的错误修复。

注册表

注册是将模组的对象 (如物品、方块、声音等) 告知游戏的过程。注册非常重要, 因为如果不注册, 游戏根本不知道这些对象的存在, 这将导致无法解释的行为和崩溃。

游戏中大多数需要注册的内容都由 Forge 注册表处理。注册表类似于一个将值映射到键的对象。Forge 使用以 ResourceLocation 为键的注册表来注册对象。这使得 ResourceLocation 可以作为对象的“注册名称”。

每种可注册对象类型都有其自己的注册表。要查看 Forge 包装的所有注册表，请参阅 ForgeRegistries 类。同一注册表中的所有注册名称必须唯一。但是，不同注册表中的名称不会冲突。例如，有一个 Block 注册表和一个 Item 注册表。一个 Block 和一个 Item 可以以相同的名称 example:thing 注册而不会冲突；但是，如果两个不同的 Block 或两个不同的 Item 以完全相同的名称注册，则第二个对象将覆盖第一个。

注册方法

有两种正确的对象注册方式：DeferredRegister 类和 RegisterEvent 生命周期事件。

DeferredRegister

DeferredRegister 是推荐的对象注册方式。它允许使用静态初始化器的便利性，同时避免了相关问题。它只是维护一个条目的供应者列表，并在 RegisterEvent 期间从这些供应者注册对象。

一个模组注册自定义方块的示例：

```
private static final DeferredRegister<Block> BLOCKS = DeferredRegister.create(ForgeRegistries.BLOCKS, MODID);

public static final RegistryObject<Block> ROCK_BLOCK = BLOCKS.register("rock", () -> new
Block(BlockBehaviour.Properties.of().mapColor(MapColor.STONE)));

public ExampleMod() {
    BLOCKS.register(FMLJavaModLoadingContext.get().getModEventBus());
}
```

RegisterEvent

RegisterEvent 是第二种注册对象的方式。此事件在模组构造方法之后、配置加载之前为每个注册表触发一次。对象通过 #register 注册，传入注册表键、注册对象名称和对象本身。还有一个额外的 #register 重载，接受一个消费辅助器来注册具有给定名称的对象。建议使用此方法以避免不必要的对象创建。

示例如下：（事件处理器注册在模组事件总线上）

```
@SubscribeEvent
public void register(RegisterEvent event) {
    event.register(ForgeRegistries.Keys.BLOCKS,
        helper -> {
            helper.register(new ResourceLocation(MODID, "example_block_1"), new Block(...));
            helper.register(new ResourceLocation(MODID, "example_block_2"), new Block(...));
            helper.register(new ResourceLocation(MODID, "example_block_3"), new Block(...));
            // ...
        }
    );
}
```

非 Forge 注册表

并非所有注册表都由 Forge 包装。这些可以是静态注册表，如 LootItemConditionType，使用是安全的。还有动态注册表，如 ConfiguredFeature 和其他一些世界生成注册表，通常以 JSON 表示。DeferredRegister#create 有一个重载，允许模组开发者指定要为其创建 RegistryObject 的原版注册表的注册表键。注册方法和附加到模组事件总线的方式与其他 DeferredRegister 相同。

!!! important 动态注册表对象**只能通过**数据文件（例如 JSON）注册。它们**不能在**代码中注册。

```
private static final DeferredRegister<LootItemConditionType> REGISTER = DeferredRegister.create(Registries.LOOT_CONDITION_TYPE,
"examplemod");

public static final RegistryObject<LootItemConditionType> EXAMPLE_LOOT_ITEM_CONDITION_TYPE =
REGISTER.register("example_loot_item_condition_type", () -> new LootItemConditionType(...));
```

!!! note 有些类本身不能注册。相反，注册的是 *Type 类，并在前者的构造方法中使用。例如，BlockEntity 有 BlockEntityType，Entity 有 EntityType。这些 *Type 类是按需创建包含类型的工厂。

这些工厂通过使用对应的 `\*Type\$Builder` 类创建。例如：（`REGISTER` 指的是一个 `DeferredRegister<BlockEntityType>`）

```

```java
public static final RegistryObject<BlockEntityType<ExampleBlockEntity>> EXAMPLE_BLOCK_ENTITY = REGISTER.register(
 "example_block_entity", () -> BlockEntityType.Builder.of(ExampleBlockEntity::new, EXAMPLE_BLOCK.get()).build(null)
);
```

```

引用已注册对象

已注册对象在创建和注册时不应存储在字段中。它们应在每次 RegisterEvent 为该注册表触发时重新创建和注册。这是为了在未来的 Forge 版本中支持模组的动态加载和卸载。

已注册对象必须始终通过 RegistryObject 或使用 @ObjectHolder 的字段来引用。

使用 RegistryObject

RegistryObject 可用于在已注册对象可用时获取对它们的引用。DeferredRegister 使用它们返回对已注册对象的引用。它们的引用在 RegisterEvent 为其注册表调用后更新，同时更新的还有 @ObjectHolder 注解。

要获取 RegistryObject，使用一个 ResourceLocation 和可注册对象的 IForgeRegistry 调用 RegistryObject#create。也可以通过提供注册表名称来使用自定义注册表。将 RegistryObject 存储在 public static final 字段中，并在需要已注册对象时调用 #get。

使用 RegistryObject 的示例：

```

public static final RegistryObject<Item> BOW = RegistryObject.create(new ResourceLocation("minecraft:bow"),
ForgeRegistries.ITEMS);

// 假设 'neomagicae:mana_type' 是一个有效的注册表，且 'neomagicae:coffeinum' 是该注册表中的一个有效对象
public static final RegistryObject<ManaType> COFFEINUM = RegistryObject.create(new ResourceLocation("neomagicae", "coffeinum"),
new ResourceLocation("neomagicae", "mana_type"), "neomagicae");

```

使用 @ObjectHolder

注册表中的已注册对象可以通过使用 @ObjectHolder 注解类或字段，并提供足够的信息来构造 ResourceLocation 以标识特定注册表中的特定对象，从而注入到 public static 字段中。

@ObjectHolder 的规则如下：

- 如果类使用 @ObjectHolder 注解，其值将是该类中所有字段的默认命名空间（如果未显式定义）
- 如果类使用 @Mod 注解，模组 ID 将是该类中所有注解字段的默认命名空间（如果未显式定义）
- 一个字段被视为注入对象需要满足以下条件：
 - 至少具有 public static 修饰符；
 - 字段使用 @ObjectHolder 注解，且：
 - * name 值已显式定义；并且
 - * registry name 值已显式定义
 - 如果字段没有对应的注册表或名称，将在编译时抛出异常
- 如果生成的 ResourceLocation 不完整或无效（路径中包含非法字符），将抛出异常
- 如果没有其他错误或异常发生，字段将被注入
- 如果上述所有规则都不适用，则不采取任何操作（可能会记录一条消息）

使用 @ObjectHolder 注解的字段在 RegisterEvent 为其注册表触发后注入值，同时注入的还有 RegistryObject。

!!! note 如果在注入时注册表中不存在该对象，将记录一条调试消息，并且不会注入任何值。

由于这些规则相当复杂，这里有一些示例：

```

class Holder {
    @ObjectHolder(registryName = "minecraft:enchantment", value = "minecraft:flame")
    public static final Enchantment flame = null;           // 注解存在。需要 [public static]。[final] 是可选的。
                                                           // 注册表名称显式定义: "minecraft:enchantment"
                                                           // 资源位置显式定义: "minecraft:flame"
                                                           // 注入内容: 来自 [Enchantment] 注册表的 "minecraft:flame"

    public static final Biome ice_flat = null;             // 字段上没有注解。
}

```

```

// 因此，该字段被忽略。

@ObjectHolder("minecraft:creeper")
public static Entity creeper = null;

// 注解存在。需要 [public static]。
// 字段上未指定注册表。
// 因此，这将产生编译时异常。

@ObjectHolder(registryName = "potion")
public static final Potion levitation = null;

// 注解存在。需要 [public static]。[final] 是可选的。
// 注册表名称显式定义: "minecraft:potion"
// 字段上未指定资源位置
// 因此，这将产生编译时异常。
}

```

创建自定义 Forge 注册表

自定义注册表通常可以只是一个简单的键到值的映射。这是一种常见的方式；但是，它强制对注册表的存在产生硬依赖。它还需要手动同步端之间的任何数据。自定义 Forge 注册表提供了一种简单的替代方案，可以实现软依赖以及更好的管理和端之间的自动同步（除非另有说明）。由于对象也使用 Forge 注册表，注册以相同的方式标准化。

自定义 Forge 注册表是使用 RegistryBuilder 创建的，可以通过 NewRegistryEvent 或 DeferredRegister。RegistryBuilder 类接受各种参数（例如注册表的名称、ID 范围以及注册表上不同事件的回调）。新的注册表在 NewRegistryEvent 完成触发后注册到 RegistryManager。

任何新创建的注册表应使用其关联的注册方法来注册关联的对象。

使用 NewRegistryEvent

当使用 NewRegistryEvent 时，使用 RegistryBuilder 调用 #create 将返回一个供应者包装的注册表。提供的注册表可以在 NewRegistryEvent 完成发布到模组事件总线后访问。在 NewRegistryEvent 完成触发之前从供应者获取自定义注册表将返回 null 值。

新数据包注册表 可以使用模组事件总线上的 DataPackRegistryEvent\$NewRegistry 事件添加新的数据包注册表。通过传入表示注册表名称的 ResourceKey 和用于从 JSON 编解码数据的 Codec，使用 #dataPackRegistry 创建注册表。可以提供可选的 Codec 用于将数据包注册表同步到客户端。

!!! important 数据包注册表不能使用 DeferredRegister 创建。它们只能通过事件创建。

使用 DeferredRegister

DeferredRegister 方法再次是对上述事件的包装。一旦使用接受注册表名称和模组 ID 的 #create 重载在常量字段中创建了 DeferredRegister，就可以通过 DeferredRegister#makeRegistry 构造注册表。这接受一个包含任何额外配置的 RegistryBuilder 供应者。该方法默认已填充 #setName。由于此方法可以随时返回，返回的是 IForgeRegistry 的供应者版本。在 NewRegistryEvent 触发之前从供应者获取自定义注册表将返回 null 值。

!!! important DeferredRegister#makeRegistry 必须在 DeferredRegister 通过 #register 添加到模组事件总线之前调用。#makeRegistry 也使用 #register 方法在 NewRegistryEvent 期间创建注册表。

处理缺失条目

在某些情况下，当模组更新或（更常见的）移除时，某些注册表对象将不再存在。可以通过第三个注册表事件 MissingMappingsEvent 指定处理缺失映射的操作。在此事件中，可以通过 #getMappings（给定注册表键和模组 ID）或 #getAllMappings（给定注册表键）获取缺失映射的列表。

!!! important MissingMappingsEvent 在 Forge 事件总线上触发。

对于每个 Mapping，可以选择以下四种映射类型之一来处理缺失条目：

| 操作 | 描述 |
|--------|-------------------|
| IGNORE | 忽略缺失条目并放弃映射。 |
| WARN | 在日志中生成警告。 |
| FAIL | 阻止世界加载。 |
| REMAP | 将条目重新映射到已注册的非空对象。 |

如果未指定任何操作，将执行默认操作，通知用户关于缺失条目的信息，并询问是否仍要加载世界。除重新映射外的所有操作将阻止任何其他注册对象占用现有 ID 的位置，以防相关条目将来重新添加到游戏中。

Minecraft 中的端

在 Minecraft 模组开发中，理解两个端 (sides) 是一个非常重要的概念：客户端和服务端。关于端存在许多常见的误解和错误，这可能导致不会使游戏崩溃，但会产生意外且常常是不可预测的后果的 Bug。

不同类型的端

当我们说“客户端”或“服务器”时，通常对我们在谈论游戏的哪个部分有一个相当直观的理解。毕竟，客户端是用户与之交互的部分，而服务器是用户为多人游戏连接的地方。很简单，对吧？

事实证明，即使是这样两个术语也可能存在歧义。以下是对“客户端”和“服务器”的四种可能含义的澄清：

- **物理客户端** —物理客户端是当你从启动器启动 Minecraft 时运行的整个程序。在游戏图形化、可交互的生命周期内运行的所有线程、进程和服务都是物理客户端的一部分。
- **物理服务器** —通常称为专用服务器 (dedicated server)，物理服务器是当你启动任何不提供可玩 GUI 的 `minecraft_server.jar` 时运行的整个程序。
- **逻辑服务器** —逻辑服务器运行游戏逻辑：生物生成、天气、更新物品栏、生命值、AI 以及所有其他游戏机制。逻辑服务器存在于物理服务器中，但它也可以在物理客户端内与逻辑客户端一起运行（即单人世界）。逻辑服务器总是在一个名为 `Server Thread` 的线程中运行。
- **逻辑客户端** —逻辑客户端接受来自玩家的输入并将其中继到逻辑服务器。此外，它还从逻辑服务器接收信息，并以图形方式呈现给玩家。逻辑客户端在 `Render Thread` 中运行，但通常还会生成其他几个线程来处理音频和区块渲染批处理等任务。

在 MinecraftForge 代码库中，物理端由名为 `Dist` 的枚举表示，而逻辑端由名为 `LogicalSide` 的枚举表示。

执行端特定操作

`Level#isClientSide`

此布尔值检查将是你最常用的端检查方式。在 `Level` 对象上查询此字段可确定该逻辑端所属的端。也就是说，如果此字段为 `true`，则该世界当前在逻辑客户端上运行。如果该字段为 `false`，则该世界在逻辑服务器上运行。因此，物理服务器始终包含 `false` 值，但我们不能假设 `false` 意味着物理服务器，因为此字段在物理客户端内的逻辑服务器上也可以为 `false`（换句话说，就是单人世界）。

每当你需要确定是否应运行游戏逻辑和其他机制时，请使用此检查。例如，如果你想在每次玩家点击你的方块时对玩家造成伤害，或者让你的机器将泥土加工成钻石，你只应在确保 `#isClientSide` 为 `false` 后执行此操作。将游戏逻辑应用于逻辑客户端，最好的情况下会导致不同步（幽灵实体、不同步的统计数据等），最坏的情况下会导致崩溃。

此检查应作为你的默认首选方法。除了 `DistExecutor` 之外，你很少需要使用其他方式来确定端和调整行为。

`DistExecutor`

考虑到为客户端和服务端模组使用单个“通用”`jar` 文件，以及将物理端分离为两个 `jar` 文件，一个重要的问题出现了：我们如何使用仅存在于一个物理端上的代码？`net.minecraft.client` 中的所有代码仅存在于物理客户端上。如果你编写的任何类以任何方式引用了这些名称，当该类在不存这些名称的环境中被加载时，游戏将会崩溃。初学者一个非常常见的错误是在方块或方块实体类中调用 `Minecraft.getInstance().<doStuff>()`，这将在类加载时立即导致任何物理服务器崩溃。

如何解决这个问题？幸运的是，FML 提供了 `DistExecutor`，它提供了各种方法，可以在不同的物理端上运行不同的方法，或者仅在某一端上运行单个方法。

!!! note 理解 FML 是基于物理端进行检查非常重要。单人世界（物理客户端内的逻辑服务器 + 逻辑客户端）始终使用 `Dist.CLIENT`！

`DistExecutor` 的工作原理是接收一个执行方法的供应者，通过利用 `invokedynamic JVM` 指令 有效防止类加载。被执行的方法应为静态方法，并且位于不同的类中。此外，如果静态方法没有参数，则应使用方法引用，而不是执行方法的供应者。

`DistExecutor` 中有两个主要方法：`#runWhenOn` 和 `#callWhenOn`。这些方法接受应执行方法的物理端以及分别运行或返回结果的供应执行方法。

这两个方法进一步细分为 `#safe*` 和 `#unsafe*` 变体。安全和 `unsafe` 变体对其用途来说都是误称。主要区别在于，在开发环境中，`#safe*` 变体会验证提供的执行方法是否为返回对另一个类的方法引用的 `lambda`，否则会抛出错误。在生产环境中，`#safe*` 和 `#unsafe*` 功能相同。


```
// 在客户端类中: ExampleClass
public static void unsafeRunMethodExample(Object param1, Object param2) {
    // ...
}

public static Object safeCallMethodExample() {
    // ...
}

// 在某个通用类中
DistExecutor.unsafeRunWhenOn(Dist.CLIENT, () -> ExampleClass.unsafeRunMethodExample(var1, var2));

DistExecutor.safeCallWhenOn(Dist.CLIENT, () -> ExampleClass::safeCallMethodExample);
```

!!! warning 由于 Java 9+ 中 `invokedynamic` 的工作方式发生了变化, `DistExecutor` 方法的所有 `#safe*` 变体在开发环境中都会将原始异常包装在 `BootstrapMethodError` 中抛出。应改用 `#unsafe*` 变体或检查 `FMLEnvironment#dist`。

线程组

如果 `Thread.currentThread().getThreadGroup() == SidedThreadGroups.SERVER` 为 `true`, 则当前线程很可能在逻辑服务器上。否则, 很可能在逻辑客户端上。当无法访问 `Level` 对象来检查 `isClientSide` 时, 这有助于获取**逻辑端**。它通过查看当前运行线程的组来猜测你所在的逻辑端。因为这是一种猜测, 此方法只应在其他选项都已用尽时使用。在几乎所有情况下, 应优先检查 `Level.isClientSide`。

FMLEnvironment#dist 和 @OnlyIn

`FMLEnvironment#dist` 保存了你的代码正在运行的**物理端**。由于它是在启动时确定的, 因此不依赖于猜测来返回结果。然而, 它的使用场景有限。

使用 `@OnlyIn(Dist)` 注解标记方法或字段, 表示加载器应完全从不在指定**物理端**上的定义中剥离该成员。通常, 只有在浏览反编译的 Minecraft 代码时才会看到这些注解, 表示 Mojang 混淆器剥离的方法。**没有**直接使用此注解的理由。请改用 `DistExecutor` 或检查 `FMLEnvironment#dist`。

常见错误

跨逻辑端访问

每当你想将信息从一个逻辑端发送到另一个逻辑端时, 你**必须**始终使用网络数据包。在单人游戏场景中, 直接将数据从逻辑服务器传输到逻辑客户端是非常诱人的做法。

这实际上通常通过静态字段无意中完成。由于在单人游戏场景中逻辑客户端和逻辑服务器共享同一个 JVM, 两个线程同时写入和读取静态字段将导致各种竞态条件和与线程相关的经典问题。

这种错误也可能显式地发生, 例如从可以在逻辑服务器上运行 (或确实运行) 的通用代码中访问物理客户端独有的类 (如 Minecraft)。初学者在物理客户端中调试时很容易忽略这个错误。代码在那里可以工作, 但在物理服务器上会立即崩溃。

编写单端模组

在最近的版本中, Minecraft Forge 已从 `mods.toml` 中移除了“端性”属性。这意味着无论你的模组是在物理客户端还是物理服务器上加载, 都应能正常工作。因此, 对于单端模组, 你通常应在 `DistExecutor#safeRunWhenOn` 或 `DistExecutor#unsafeRunWhenOn` 中注册事件处理器, 而不是在模组构造方法中直接调用相关的注册方法。基本上, 如果你的模组在错误的端上加载, 它应该什么都不做, 不监听任何事件, 等等。单端模组本质上不应注册方块、物品等, 因为它们在另一端也需要可用。

此外, 如果你的模组是单端的, 它通常不应阻止用户加入缺少该模组的服务器。因此, 你应该将 `mods.toml` 中的 `displayTest` 属性设置为必要的值。

```
[[mods]]
# ...

# MATCH_VERSION 意味着如果你的模组在客户端和服务端上的版本不同, 将显示红 X。这是默认行为, 如果你的模组同时有服务器和客户端元素, 应选择此选项。
# IGNORE_SERVER_VERSION 意味着如果模组存在于服务器但不在客户端上, 不会显示红 X。如果你只是服务器端模组, 应使用此选项。
# IGNORE_ALL_VERSION 意味着如果模组存在于客户端或服务端上, 不会显示红 X。这是一种特殊情况, 仅在你的模组没有服务器组件时应使用。
# NONE 表示你的模组未设置显示测试。你需要自行设置, 更多信息请参阅 IExtensionPoint.DisplayTest。使用此值你可以定义任何你希望的方案。
# 重要说明: 这不是关于你的模组在哪些环境 (CLIENT 或 DEDICATED SERVER) 中加载的指示。你的模组应该在你所在的任何地方加载 (并且可能什么都不做!)。
displayTest="IGNORE_ALL_VERSION" # 如果未指定, MATCH_VERSION 是默认值 (可选)
```

如果要使用自定义显示测试，则应将 `displayTest` 选项设置为 `NONE`，并注册 `IExtensionPoint$DisplayTest` 扩展：

```
// 确保模组在另一方网络侧缺失时，不会导致客户端显示服务器不兼容
ModLoadingContext.get().registerExtensionPoint(IExtensionPoint.DisplayTest.class, () -> new IExtensionPoint.DisplayTest(() ->
NetworkConstants.IGNORESERVERONLY, (a, b) -> true));
```

这告诉客户端，它应忽略服务器版本缺失，并告诉服务器，它不应告诉客户端此模组应存在。因此，此代码片段同时适用于仅客户端和仅服务器端的模组。

事件

Forge 使用事件总线 (Event Bus) 让模组能够拦截原版和各种模组行为中的事件。

例如：可以在右键点击原版木棍时，通过事件来执行某个操作。

大多数事件使用的主事件总线位于 `MinecraftForge#EVENT_BUS`。还有一个用于模组特定事件的事件总线，位于 `FMLJavaModLoadingContext#getModEventBus`，仅在特定情况下使用。更多关于此总线的信息见下文。

每个事件都在其中一个总线上触发：大多数事件在主 Forge 事件总线上触发，但有些事件在模组特定的事件总线上触发。

事件处理器是已注册到事件总线上的某个方法。

创建事件处理器

事件处理器方法接受单个参数且不返回结果。方法可以是静态的或实例方法，具体取决于实现。

事件处理器可以直接使用 `IEventBus#addListener` 注册，泛型事件（以继承 `GenericEvent<T>` 表示）则使用 `IEventBus#addGenericListener` 注册。这两种监听器添加方法都接受一个表示方法引用的消费者。泛型事件处理器还需要指定泛型的类。事件处理器必须在主模组类的构造方法中注册。

```
// 在主模组类 ExampleMod 中

// 此事件在模组总线上
private void modEventHandler(RegisterEvent event) {
    // 在这里处理
}

// 此事件在 Forge 总线上
private static void forgeEventHandler(AttachCapabilitiesEvent<Entity> event) {
    // ...
}

// 在模组构造方法中
modEventBus.addListener(this::modEventHandler);
forgeEventBus.addGenericListener(Entity.class, ExampleMod::forgeEventHandler);
```

实例注解事件处理器

此事件处理器监听 `EntityItemPickupEvent`，顾名思义，当某个 `Entity` 拾起物品时会发布到事件总线。

```
public class MyForgeEventHandler {
    @SubscribeEvent
    public void pickupItem(EntityItemPickupEvent event) {
        System.out.println(" 物品被拾取! ");
    }
}
```

要注册此事件处理器，使用 `MinecraftForge.EVENT_BUS.register(...)` 并传入事件处理器所在类的实例。如果要注册到模组特定的事件总线，应使用 `FMLJavaModLoadingContext.get().getModEventBus().register(...)`。

静态注解事件处理器

事件处理器也可以是静态的。处理方法仍然使用 `@SubscribeEvent` 注解。与实例处理器的唯一区别在于它被标记为 `static`。要注册静态事件处理器，传入类的实例是不够的，必须传入 `Class` 本身。例如：

```
public class MyStaticForgeEventHandler {
    @SubscribeEvent
    public static void arrowNocked(ArrowNockEvent event) {
        System.out.println(" 搭箭! ");
    }
}
```

必须这样注册: `MinecraftForge.EVENT_BUS.register(MyStaticForgeEventHandler.class)`。

自动注册静态事件处理器

类可以使用 `@Mod$EventBusSubscriber` 注解进行标注。这样的类会在 `@Mod` 类本身构造完成后自动注册到 `MinecraftForge#EVENT_BUS`。这本质上相当于在 `@Mod` 类构造方法的末尾添加了 `MinecraftForge.EVENT_BUS.register(AnnotatedClass.class);`。

可以向 `@Mod$EventBusSubscriber` 注解传入你想要监听的总线。建议同时指定模组 ID (因为注解处理过程可能无法自动推断) 和你注册到的总线 (作为提醒, 确保你在正确的总线上)。你还可以指定 `Dist` 或物理端 (physical sides) 来加载此事件订阅器。这可以用于不在专用服务器上加载客户端特有的事件订阅器。

以下是一个监听 `RenderLevelStageEvent` 的静态事件监听器示例, 它仅在客户端被调用:

```
@Mod.EventBusSubscriber(modid = "mymod", bus = Bus.FORGE, value = Dist.CLIENT)
public class MyStaticClientOnlyEventHandler {
    @SubscribeEvent
    public static void drawLast(RenderLevelStageEvent event) {
        System.out.println(" 绘制中! ");
    }
}
```

!!! note 这不会注册类的实例, 而是注册类本身 (即事件处理方法必须是静态的)。

取消事件

如果一个事件可以被取消, 它会被标记为 `@Cancelable` 注解, 并且 `Event#isCancelable()` 方法会返回 `true`。可取消事件的取消状态可以通过调用 `Event#setCanceled(boolean canceled)` 来修改, 传入 `true` 表示取消事件, 传入 `false` 表示“取消取消”事件。但是, 如果事件不可取消 (由 `Event#isCancelable()` 定义), 无论传入什么布尔值, 都会抛出 `UnsupportedOperationException`, 因为不可取消事件的取消状态被视为不可变的。

!!! important 并非所有事件都可以被取消! 尝试取消一个不可取消的事件将导致抛出未检查的 `UnsupportedOperationException`, 这预计会导致游戏崩溃! 在尝试取消事件之前, 始终使用 `Event#isCancelable()` 检查事件是否可取消!

结果

有些事件具有 `Event$Result`。结果可以是以下三种之一: `DENY` 阻止事件, `DEFAULT` 使用原版行为, `ALLOW` 强制采取行动, 无论原本是否会发生。事件的结果可以通过在事件上调用 `#setResult` 并传入 `Event$Result` 来设置。并非所有事件都有结果; 有结果的事件会使用 `@HasResult` 注解标注。

!!! important 不同的事件可能以不同方式使用结果, 在使用结果前请参考事件的 `JavaDoc`。

优先级

事件处理器方法 (使用 `@SubscribeEvent` 标记) 具有优先级。可以通过设置注解的 `priority` 值来设定事件处理器方法的优先级。优先级可以是 `EventPriority` 枚举中的任意值 (`HIGHEST`、`HIGH`、`NORMAL`、`LOW` 和 `LOWEST`)。优先级为 `HIGHEST` 的事件处理器最先执行, 然后按降序执行, 直到最后执行 `LOWEST` 事件。

子事件

许多事件具有不同的变体。这些变体可能不同, 但都基于一个共同因素 (例如 `PlayerEvent`), 或者可能是具有多个阶段的事件 (例如 `PotionBrewEvent`)。请注意, 如果你监听父事件类, 你的方法将为**所有**子类接收调用。

模组事件总线

模组事件总线主要用于监听模组应进行初始化的生命周期事件。模组总线上的每个事件都需要实现 `IModBusEvent`。其中许多事件也是并行运行的，这样多个模组可以同时初始化。这意味着你不能在这些事件中直接执行来自其他模组的代码。请使用 `InterModComms` 系统来实现这一点。

以下是在模组初始化期间，模组事件总线上最常用的四个生命周期事件：

- `FMLCommonSetupEvent`
- `FMLClientSetupEvent` 和 `FMLDedicatedServerSetupEvent`
- `InterModEnqueueEvent`
- `InterModProcessEvent`

!!! note `FMLClientSetupEvent` 和 `FMLDedicatedServerSetupEvent` 仅在各自对应的分发端 (distribution) 上被调用。

这四个生命周期事件都是并行运行的，因为它们都是 `ParallelDispatchEvent` 的子类。如果希望在任意 `ParallelDispatchEvent` 期间在主线程上运行代码，可以使用 `#enqueueWork`。

除了生命周期事件外，还有一些在模组事件总线上触发的杂项事件，用于注册、设置或初始化各种内容。与生命周期事件不同，这些事件大多不是并行运行的。几个例子：

- `RegisterColorHandlersEvent`
- `ModelEvent$BakingCompleted`
- `TextureStitchEvent`
- `RegisterEvent`

一个好的经验法则是：当事件应该在模组初始化期间处理时，它们在模组事件总线上触发。

模组生命周期

在模组加载过程中，各种生命周期事件会在模组特定的事件总线上触发。许多操作在这些事件期间执行，例如注册对象、准备数据生成或与其他模组通信。

事件监听器应使用 `@EventBusSubscriber(bus = Bus.MOD)` 或在模组构造方法中注册：

```
@Mod.EventBusSubscriber(modid = "mymod", bus = Mod.EventBusSubscriber.Bus.MOD)
public class MyModEventSubscriber {
    @SubscribeEvent
    static void onCommonSetup(FMLCommonSetupEvent event) { ... }
}

@Mod("mymod")
public class MyMod {
    public MyMod() {
        FMLModLoadingContext.get().getModEventBus().addListener(this::onCommonSetup);
    }

    private void onCommonSetup(FMLCommonSetupEvent event) { ... }
}
```

!!! warning 大多数生命周期事件是并行触发的：所有模组将同时接收到相同的事件。

模组**必须**注意线程安全，例如在调用其他模组的 API 或访问原版系统时。通过 `ParallelDispatchEvent#enqueueWork` 将代码排入队列。

注册事件

注册事件在模组实例构造之后触发。共有三个：`NewRegistryEvent`、`DataPackRegistryEvent$NewRegistry` 和 `RegisterEvent`。这些事件在模组加载期间同步触发。

`NewRegistryEvent` 允许模组开发者使用 `RegistryBuilder` 类注册自定义注册表。

`DataPackRegistryEvent$NewRegistry` 允许模组开发者通过提供 `Codec` 来编解码 JSON 中的对象，从而注册自定义数据包注册表。

`RegisterEvent` 用于注册对象到注册表中。该事件会为每个注册表触发一次。

数据生成

如果游戏设置为运行数据生成器，那么 `GatherDataEvent` 将是最后触发的事件。此事件用于将模组的数据提供者注册到关联的数据生成器。此事件也是同步触发的。

通用设置

`FMLCommonSetupEvent` 用于物理客户端和服务端通用的操作，例如注册能力 (capabilities)。

端侧设置

端侧设置事件在各自的物理端上触发：`FMLClientSetupEvent` 在物理客户端上触发，`FMLDedicatedServerSetupEvent` 在专用服务器上触发。这是进行物理端特定初始化（例如注册客户端按键绑定）的地方。

模组间通信 (InterModComms)

这是用于跨模组兼容性而向模组发送消息的地方。有两个事件：`InterModEnqueueEvent` 和 `InterModProcessEvent`。

`InterModComms` 是负责保存模组消息的类。这些方法在生命周期事件期间调用是安全的，因为它由 `ConcurrentMap` 支持。

在 `InterModEnqueueEvent` 期间，使用 `InterModComms#sendTo` 向不同模组发送消息。这些方法接受将接收消息的模组 ID、消息数据关联的键以及包含消息数据的供应者 (supplier)。此外，也可以指定消息的发送者，但默认情况下会是调用者的模组 ID。

然后在 `InterModProcessEvent` 期间，使用 `InterModComms#getMessages` 获取所有接收到的消息的流。提供的模组 ID 几乎总是调用该方法的模组的模组 ID。此外，可以指定一个谓词来过滤消息键。这将返回一个 `IMCMessage` 流，其中包含数据的发送者、数据的接收者、数据键以及提供的实际数据。

!!! note 还有另外两个生命周期事件：`FMLConstructModEvent`，在模组实例构造之后、`RegisterEvent` 之前触发；以及 `FMLLoadCompleteEvent`，在 `InterModComms` 事件之后、模组加载过程完成时触发。

资源

资源是游戏使用的额外数据，存储在数据文件中，而不是代码中。Minecraft 有两个主要的资源系统：逻辑客户端上的资源系统用于视觉内容，如模型、纹理和本地化，称为 `assets`；逻辑服务器上的资源系统用于游戏玩法内容，如配方和战利品表，称为 `data`。资源包控制前者，而数据包控制后者。

在默认模组开发套件中，`assets` 和 `data` 目录位于项目的 `src/main/resources` 目录下。

当启用多个资源包或数据包时，它们会被合并。通常，包栈顶部的文件会覆盖下面的文件；但对于某些文件，如本地化文件和标签，数据实际上会进行内容层面的合并。模组在其 `resources` 目录中定义资源和数据包，但它们被视为“模组资源”包的子集。模组资源包不能被禁用，但可以被其他资源包覆盖。模组数据包可以使用原版的 `/datapack` 命令禁用。

所有资源应使用蛇形命名法 (snake case) 的路径和文件名 (小写，使用 “_” 分隔单词)，这在 1.11 及以上版本中是强制要求的。

ResourceLocation

Minecraft 使用 `ResourceLocation` 来标识资源。一个 `ResourceLocation` 包含两部分：命名空间 (namespace) 和路径 (path)。它通常指向 `assets/<namespace>/<ctx>/<path>` 处的资源，其中 `ctx` 是取决于 `ResourceLocation` 使用方式的上下文相关路径片段。当 `ResourceLocation` 以字符串形式写入/读取时，它表示为 `<namespace>:<path>`。如果省略命名空间和冒号，则在将字符串读入 `ResourceLocation` 时，命名空间将默认为 “minecraft”。模组应将其资源放入与其模组 ID 同名的命名空间中（例如，ID 为 `examplemod` 的模组应将其资源分别放入 `assets/examplemod` 和 `data/examplemod`，指向这些文件的 `ResourceLocation` 看起来像 `examplemod:<path>`）。这不是强制要求，在某些情况下，使用不同的（甚至多个）命名空间可能更可取。`ResourceLocation` 也在资源系统之外使用，因为它恰好是唯一标识对象的好方法（例如注册表）。

国际化与本地化

国际化 (Internationalization, 简称 `i18n`) 是一种设计代码的方式，使其无需更改即可适应多种语言。本地化 (Localization) 是将显示的文本适应用户语言的过程。

I18n 通过 **翻译键 (translation keys)** 实现。翻译键是一个字符串，用于标识一段不特定于某种语言的显示文本。例如，`block.minecraft.dirt` 是指草方块名称的翻译键。这样，显示文本可以在不关心特定语言的情况下被引用。代码无需更改即可适应新语言。

本地化将在游戏的语言环境中进行。在 Minecraft 客户端中，语言环境由语言设置指定。在专用服务器上，唯一支持的语言环境是 `en_us`。可用的语言环境列表可以在 Minecraft Wiki 上找到。

语言文件

语言文件位于 `assets/[namespace]/lang/[locale].json` (例如，`examplemod` 提供的所有美式英语翻译都在 `assets/examplemod/lang/en_us.json` 中)。文件格式是一个从翻译键到值的 JSON 映射。文件必须使用 UTF-8 编码。旧的 `.lang` 文件可以使用转换器转换为 json。

```
{
  "item.examplemod.example_item": "Example Item Name",
  "block.examplemod.example_block": "Example Block Name",
  "commands.examplemod.examplecommand.error": "Example Command Errored!"
}
```

在方块和物品中的使用

Block、Item 以及其他一些 Minecraft 类具有用于显示其名称的内置翻译键。这些翻译键通过重写 `#getDescriptionId` 来指定。Item 还有 `#getDescriptionId(ItemStack)`，可以重写以根据 `ItemStack` NBT 提供不同的翻译键。

默认情况下，`#getDescriptionId` 会返回 `block.` 或 `item.` 后接方块或物品的注册名称，冒号替换为点。BlockItem 默认重写了此方法以使用其对应 Block 的翻译键。例如，一个 ID 为 `examplemod:example_item` 的物品实际上需要语言文件中包含以下行：

```
{
  "item.examplemod.example_item": "Example Item Name"
}
```

!!! note 翻译键的唯一用途是国际化。不要将其用于逻辑判断。请使用注册名称。

本地化方法

!!! warning 一个常见的问题是让服务器为客户端进行本地化。服务器只能在其自身的语言环境中进行本地化，这不一定会与连接的客户端的语言环境匹配。

为了尊重客户端的语言设置，服务器应让客户端使用 `TranslatableComponent` 或其他保留语言中立翻译键的方法，在其自身的语言环境中进行本地化。

net.minecraft.client.resources.language.I18n (仅客户端)

此 I18n 类仅在 Minecraft 客户端上存在！ 它旨在供仅在客户端运行的代码使用。在服务器上尝试使用它会抛出异常并崩溃。

- `get(String, Object...)` 在客户端的语言环境中进行本地化并支持格式化。第一个参数是翻译键，其余参数是 `String.format(String, Object...)` 的格式化参数。

TranslatableContents

`TranslatableContents` 是一个惰性本地化和格式化的 `ComponentContents`。它在向玩家发送消息时非常有用，因为它会自动在玩家自己的语言环境中进行本地化。

`TranslatableContents(String, Object...)` 构造方法的第一个参数是翻译键，其余参数用于格式化。唯一支持的格式说明符是 `%s` 和 `%1$s`、`%2$s`、`%3$s` 等。格式化参数可以是 `Component`，它们将插入到最终的格式化文本中，并保留其所有属性。

`MutableComponent` 可以通过传入 `TranslatableContents` 的参数，使用 `Component#translatable` 创建。也可以通过传入 `ComponentContents` 本身，使用 `MutableComponent#create` 创建。

TextComponentHelper

- `createComponentTranslation(CommandSource, String, Object...)` 根据接收者创建一个本地化并格式化的 `MutableComponent`。如果接收者是原版客户端，则立即进行本地化和格式化。如果不是，则使用包含 `TranslatableContents` 的 `Component` 进行惰性本地化和格式化。这仅在服务器应允许原版客户端连接时有用。

方块

方块显然是 Minecraft 世界的基础。它们构成了所有地形、结构和机械。如果你有兴趣制作模组，那么你很可能会想要添加一些方块。本页面将指导你完成方块的创建，以及你可以用它们做的一些事情。

创建方块

基础方块

对于简单的方块，不需要特殊功能（比如圆石、木板等），则不需要自定义类。你可以通过使用 `BlockBehaviour$Properties` 对象实例化 `Block` 类来创建方块。这个 `BlockBehaviour$Properties` 对象可以使用 `BlockBehaviour$Properties#of` 创建，并可以通过调用其方法进行自定义。例如：

- `strength` - 硬度控制破坏方块所需的时间。这是一个任意值。作为参考，石头的硬度为 1.5，泥土为 0.5。如果方块应该是不可破坏的，则应将硬度设为 -1.0，参见 `Blocks#BEDROCK` 的定义作为示例。抗性控制方块的爆炸抗性。作为参考，石头的抗性为 6.0，泥土为 0.5。
- `sound` - 控制方块在被击打、破坏或放置时发出的声音。需要 `SoundType` 参数，更多详情请参见声音页面。
- `lightLevel` - 控制方块的发光亮度。接受一个以 `BlockState` 为参数、返回 0 到 15 之间值的函数。
- `friction` - 控制方块的滑动程度。作为参考，冰的滑动系数为 0.98。

所有这些方法都是**可链式调用**的，这意味着你可以按顺序调用它们。有关示例，请参见 `Blocks` 类。

!!! note 方块没有设置其 `CreativeModeTab` 的设值方法。如果方块有关联的物品（例如 `BlockItem`），这由 `BuildCreativeModeTabContentsEvent` 处理。此外，方块也没有翻译键的设值方法，因为翻译键是通过注册表名称经由 `Block#getDescriptionId` 自动生成的。

高级方块

当然，上述方法只允许创建非常基础的方块。如果你想要添加功能，比如玩家交互，就需要自定义类。然而，`Block` 类有许多方法，不幸的是无法在此——记录。关于你可以用方块做的事情，请参见本节中的其他页面。

注册方块

方块必须被注册才能正常工作。

!!! important 世界中的方块和物品栏中的“方块”是完全不同的概念。世界中的方块由 `BlockState` 表示，其行为由 `Block` 实例定义。与此同时，物品栏中的物品是 `ItemStack`，由 `Item` 控制。作为连接 `Block` 和 `Item` 这两个不同世界的桥梁，存在着 `BlockItem` 类。`BlockItem` 是 `Item` 的子类，它有一个 `block` 字段，持有对其所代表的 `Block` 的引用。`BlockItem` 定义了“方块”作为物品时的一些行为，比如右键点击放置方块。一个 `Block` 可以没有对应的 `BlockItem`（例如 `minecraft:water` 作为方块存在，但不作为物品存在，因此无法在物品栏中持有它）。

当注册一个方块时，*仅仅*注册了一个方块。该方块并不会自动拥有 `BlockItem`。要为方块创建一个基本的 `BlockItem`，应将 `BlockItem` 注册为 `Block` 的 `BlockItem`。

可选注册方块 过去有一些模组允许用户通过配置文件禁用方块/物品。但是，你不应该这样做。可注册的方块数量没有限制，因此请注册你模组中的所有方块！如果你希望通过配置文件禁用某个方块，你应该禁用其合成配方。如果你希望在创造模式标签页中禁用该方块，请在 `BuildCreativeModeTabContentsEvent` 中构建内容时使用 `FeatureFlag`。

延伸阅读

关于方块属性的信息，例如原版中用于栅栏、墙等方块的属性，请参见方块状态章节。

方块状态

遗留行为

在 Minecraft 1.7 及更早版本中，需要存储放置或状态数据但没有 `BlockEntity` 的方块使用**元数据**。元数据是与方块一起存储的额外数字，允许在同一方块内实现不同的旋转、朝向甚至完全独立的行为。

然而，元数据系统令人困惑且有限，因为它仅仅是作为数字与方块 ID 一起存储，除了代码中的注释外没有任何意义。例如，要实现一个可以朝向某个方向并且可以位于方块空间上半部分或下半部分的方块（如楼梯）：


```
switch (meta) {
    case 0: { ... } // 朝南, 位于方块下半部分
    case 1: { ... } // 朝南, 位于方块上半部分
    case 2: { ... } // 朝北, 位于方块下半部分
    case 3: { ... } // 朝北, 位于方块上半部分
    // ... 以此类推 ...
}
```

因为这些数字本身没有任何意义, 除非拥有源代码和注释, 否则没有人知道它们代表什么。

状态的引入

在 Minecraft 1.8 及以上版本中, 元数据系统和方块 ID 系统已被弃用, 并最终被**方块状态系统**取代。方块状态系统将方块属性的细节从方块的其他行为中抽象出来。

方块的每个属性由 `Property<?>` 实例描述。方块属性的例子包括乐器 (`EnumProperty<NoteBlockInstrument>`)、朝向 (`DirectionProperty`)、是否被激活 (`Property<Boolean>`) 等。每个属性具有由 `Property<T>` 参数化的类型 `T` 的值。

可以从 `Block` 和从 `Property<?>` 到其关联值的映射构造一个唯一的配对。这个唯一的配对称为 `BlockState`。

先前无意义的元数据值系统已被方块属性系统取代, 后者更易于解释和处理。以前, 一个朝东且被激活或按下的石质按钮表示为 “minecraft:stone_button, 元数据为 9”。现在, 这表示为 “minecraft:stone_button[facing=east,powered=true]”。

方块状态的正确使用

`BlockState` 系统是一个灵活且强大的系统, 但它也有局限性。`BlockState` 是不可变的, 并且其所有属性的组合在游戏启动时生成。这意味着拥有具有许多属性和可能值的 `BlockState` 会减慢游戏的加载速度, 并使试图理解你的方块逻辑的人感到困惑。

并非所有方块和情况都需要使用 `BlockState`; 只有方块最基本的属性才应放入 `BlockState`, 而其他情况更适合使用 `BlockEntity` 或作为单独的 `Block`。始终考虑你是否真的需要为你的目的使用方块状态。

!!! note 一个好的经验法则是: **如果它有不同名称, 则应是一个单独的方块。**

一个例子是制作椅子方块: 椅子的方向应是一个属性, 而不同的木材种类应分为不同的方块。一个朝东的“橡木椅子” (`oak_chair[facing=east]`) 与一个朝西的“云杉椅子” (`spruce_chair[facing=west]`) 是不同的。

实现方块状态

在你的 `Block` 类中, 为你 `Block` 的每个属性创建或引用 `static final` 的 `Property<?>` 对象。你可以自由创建自己的 `Property<?>` 实现, 但本文不涉及具体的实现方法。原版代码提供了几个便利的实现:

- `IntegerProperty`
 - 实现 `Property<Integer>`。定义一个持有整数值属性的。
 - 通过调用 `IntegerProperty#create(String propertyName, int minimum, int maximum)` 创建。
- `BooleanProperty`
 - 实现 `Property<Boolean>`。定义一个持有 `true` 或 `false` 值的属性。
 - 通过调用 `BooleanProperty#create(String propertyName)` 创建。
- `EnumProperty<E extends Enum<E>>`
 - 实现 `Property<E>`。定义一个可以取枚举类值的属性。
 - 通过调用 `EnumProperty#create(String propertyName, Class<E> enumClass)` 创建。
 - 也可以只使用枚举值的一个子集 (例如 16 种 `DyeColor` 中的 4 种)。参见 `EnumProperty#create` 的重载方法。
- `DirectionProperty`
 - 这是 `EnumProperty<Direction>` 的便利实现。
 - 还提供了几个便利的谓词。例如, 要获取表示基本方向的属性, 调用 `DirectionProperty.create("<name>", Direction.Plane.HORIZONTAL)`; 要获取 X 轴方向, 调用 `DirectionProperty.create("<name>", Direction.Axis.X)`。

`BlockStateProperties` 类包含了共享的原版属性, 应尽可能使用或引用它们, 而不是创建你自己的属性。

当你有了所需的 `Property<>` 对象后, 在你的 `Block` 类中重写 `Block#createBlockStateDefinition(StateDefinition$Builder)`。在该方法中, 调用 `StateDefinition$Builder#add(...)`; 参数为你希望方块拥有的每个 `Property<?>`。

每个方块还会有一个自动为你选择的“默认”状态。你可以通过在构造函数中调用 `Block#registerDefaultState(BlockState)` 方法来更改这个“默认”状态。当你的方块被放置时, 它将变为这个“默认”状态。来自 `DoorBlock` 的一个示例:


```

this.registerDefaultState(
    this.stateDefinition.any()
        .setValue(FACING, Direction.NORTH)
        .setValue(OPEN, false)
        .setValue(HINGE, DoorHingeSide.LEFT)
        .setValue(POWERED, false)
        .setValue(HALF, DoubleBlockHalf.LOWER)
);

```

如果你希望更改放置方块时使用的 `BlockState`，可以重写 `Block#getStateForPlacement(BlockPlaceContext)`。例如，这可以用来根据玩家放置方块时所处的位置来设置方块的方向。

由于 `BlockState` 是不可变的，并且所有属性组合都在游戏启动时生成，调用 `BlockState#setValue(Property<T>, T)` 将简单地转到 `Block` 的 `StateHolder` 并请求具有你想要的值组合的 `BlockState`。

由于所有可能的 `BlockState` 都在启动时生成，你可以并且被鼓励使用引用相等运算符 (`==`) 来检查两个 `BlockState` 是否相等。

使用 BlockState

你可以通过调用 `BlockState#getValue(Property<?>)` 来获取属性的值，传入你想要获取值的属性。如果你想要获取具有不同值集的 `BlockState`，只需调用 `BlockState#setValue(Property<T>, T)`，传入属性和其值。

你可以使用 `Level#setBlockAndUpdate(BlockPos, BlockState)` 和 `Level#getBlockState(BlockPos)` 在世界中获取和放置 `BlockState`。如果你要放置一个 `Block`，调用 `Block#defaultBlockState()` 获取“默认”状态，然后按照上述方法使用 `BlockState#setValue(Property<T>, T)` 来实现所需的状态。

方块实体

`BlockEntity` (方块实体) 类似于绑定到方块的简化的 `Entity` (实体)。它们用于存储动态数据、执行基于 `tick` 的任务以及进行动态渲染。原版 `Minecraft` 中的一些例子包括：箱子中物品栏的处理、熔炉的烧炼逻辑、信标的效果范围。模组中还有更高级的例子，如采石场、分拣机、管道和显示器。

!!! note `BlockEntity` 并非万能的解决方案，使用不当会导致卡顿。尽可能避免使用它们。

注册

方块实体是动态创建和移除的，因此它们本身并不是注册表对象。

为了创建一个 `BlockEntity`，你需要继承 `BlockEntity` 类。相应地，会注册另一个对象来轻松创建和引用动态对象的类型。对于 `BlockEntity`，这些被称为 `BlockEntityType`。

`BlockEntityType` 可以像任何其他注册表对象一样被注册。要构造一个 `BlockEntityType`，可以通过 `BlockEntityType$Builder#of` 使用其构建器形式。该方法接受两个参数：一个 `BlockEntityType$BlockEntitySupplier` (接受 `BlockPos` 和 `BlockState` 来创建关联 `BlockEntity` 的新实例)，以及一个可变参数的 `Block` 数组，表示此 `BlockEntity` 可以附加到的方块。通过调用 `BlockEntityType$Builder#build` 来完成 `BlockEntityType` 的构建。这需要一个 `Type`，表示在 `DataFixer` 中用于引用此注册表对象的类型安全引用。由于 `DataFixer` 对于模组来说是一个可选系统，因此可以传入 `null`。

```

// 对于某个 DeferredRegister<BlockEntityType<?>> REGISTER
public static final RegistryObject<BlockEntityType<MyBE>> MY_BE = REGISTER.register("mybe", () ->
    BlockEntityType.Builder.of(MyBE::new, validBlocks).build(null));

// 在 MyBE 中，一个 BlockEntity 的子类
public MyBE(BlockPos pos, BlockState state) {
    super(MY_BE.get(), pos, state);
}

```

创建 BlockEntity

要创建 `BlockEntity` 并将其附加到 `Block`，必须在你 `Block` 的子类上实现 `EntityBlock` 接口。必须实现 `EntityBlock#newBlockEntity(BlockPos, BlockState)` 方法并返回你的 `BlockEntity` 的新实例。

在 BlockEntity 中存储数据

为了保存数据，请重写以下两个方法：

```
BlockEntity#saveAdditional(CompoundTag tag)

BlockEntity#load(CompoundTag tag)
```

每当包含 BlockEntity 的 LevelChunk 从标签加载或保存到标签时，都会调用这些方法。使用它们来读取和写入你的方块实体类中的字段。

!!! note 每当你的数据发生变化时，你需要调用 BlockEntity#setChanged；否则，包含你 BlockEntity 的 LevelChunk 可能会在保存世界时被跳过。

!!! important 务必调用父类的 super 方法！

`super` 方法保留了以下标签名称：`id`、`x`、`y`、`z`、`ForgeData` 和 `ForgeCaps`。

使 BlockEntity 进行 tick

如果你需要一个进行 tick 的 BlockEntity，例如跟踪烧炼过程中的进度，则必须在 EntityBlock 中实现并重写另一个方法：EntityBlock#getTicker(Level, BlockState, BlockEntityType)。这可以根据玩家所处的逻辑侧实现不同的 tick 方法，或者只实现一个通用的 tick 方法。无论哪种情况，都必须返回一个 BlockEntityTicker。由于这是一个函数式接口，因此可以直接传入一个表示 tick 方法的方法引用：

```
// 在某个 Block 子类中
@Nullable
@Override
public <T extends BlockEntity> BlockEntityTicker<T> getTicker(Level level, BlockState state, BlockEntityType<T> type) {
    return type == MyBlockEntityTypes.MYBE.get() ? MyBlockEntity::tick : null;
}

// 在 MyBlockEntity 中
public static void tick(Level level, BlockPos pos, BlockState state, MyBlockEntity blockEntity) {
    // 执行操作
}
```

!!! note 此方法每个 tick 都会被调用；因此，应避免在其中进行复杂计算。如果可能，你应该每 X 个 tick 执行一次更复杂的计算。（每秒的 tick 数可能低于 20（二十），但不会更高）

将数据同步到客户端

有三种方法可以将数据同步到客户端：在区块加载时同步、在方块更新时同步以及使用自定义网络消息同步。

在 LevelChunk 加载时同步

为此，你需要重写

```
BlockEntity#getUpdateTag()

IForgeBlockEntity#handleUpdateTag(CompoundTag tag)
```

同样，这相当简单，第一个方法收集应发送到客户端的数据，而第二个方法处理该数据。如果你的 BlockEntity 包含的数据不多，你可能可以直接使用在 BlockEntity 中存储数据章节中的方法。

!!! important 为方块实体同步过多/无用的数据会导致网络拥塞。你应该优化网络使用，只发送客户端需要的信息，并在客户端需要时发送。例如，在更新标签中发送方块实体的物品栏通常是不必要的，因为这可以通过其 AbstractContainerMenu 进行同步。

在方块更新时同步

这种方法稍微复杂一些，但你仍然只需要重写两个或三个方法。以下是一个简单的实现示例：

```
@Override
public CompoundTag getUpdateTag() {
    CompoundTag tag = new CompoundTag();
    // 将你的数据写入标签
    return tag;
}
```

```
@Override
public Packet<ClientGamePacketListener> getUpdatePacket() {
    // 将从 #getUpdateTag 获取标签
    return ClientboundBlockEntityDataPacket.create(this);
}

// 可以重写 IForgeBlockEntity#onDataPacket。默认情况下，这将委托给 #Load 方法。
```

静态构造方法 `ClientboundBlockEntityDataPacket#create` 接受：

- `BlockEntity`。
- 一个可选函数，用于从 `BlockEntity` 获取 `CompoundTag`。默认情况下，使用 `BlockEntity#getUpdateTag`。

现在，要发送数据包，必须在服务器端发送更新通知。

```
Level#sendBlockUpdated(BlockPos pos, BlockState oldState, BlockState newState, int flags)
```

`pos` 应为你的 `BlockEntity` 的位置。对于 `oldState` 和 `newState`，你可以传入该位置的当前 `BlockState`。`flags` 是一个位掩码，应包含 2，这会将更改同步到客户端。更多信息以及其余标志请参见 `Block`。标志 2 等同于 `Block#UPDATE_CLIENTS`。

使用自定义网络消息同步

这种同步方式可能是最复杂的，但通常是最优化的，因为你可以确保只有需要同步的数据才会被实际同步。在尝试此方法之前，你应该先查看网络章节，特别是 `SimpleImpl`。一旦你创建了自定义网络消息，就可以使用 `SimpleChannel#send(PacketDistributor$PacketTarget, MSG)` 将其发送给所有加载了该 `BlockEntity` 的玩家。

!!! warning 务必进行安全检查，当消息到达玩家时，`BlockEntity` 可能已经被销毁/替换！你还应检查区块是否已加载 (`Level#hasChunkAt(BlockPos)`)。

方块实体渲染器

`BlockEntityRenderer`（方块实体渲染器，简称 BER）用于渲染无法用静态烘焙模型（JSON、OBJ、B3D 等）表示的方块。方块实体渲染器要求方块拥有 `BlockEntity`。

创建 BER

要创建 BER，请创建一个继承自 `BlockEntityRenderer` 的类。它接受一个泛型参数来指定方块的 `BlockEntity` 类。该泛型参数在 BER 的 `render` 方法中使用。

对于给定的 `BlockEntityType`，只存在一个 BER。因此，特定于世界中单个实例的值应存储在传递给渲染器的方块实体中，而不是存储在 BER 本身中。例如，一个每帧递增的整数，如果存储在 BER 中，将会为世界中此类型的所有方块实体每帧递增。

render

此方法每帧被调用，用于渲染方块实体。

参数

- `blockEntity`：正在被渲染的方块实体实例。
- `partialTick`：自上一个完整 tick 以来经过的时间，以 tick 的分数表示。
- `poseStack`：一个持有四维矩阵条目的堆栈，偏移到方块实体的当前位置。
- `bufferSource`：能够访问顶点消费者的渲染缓冲区。
- `combinedLight`：方块实体上当前光照值的整数。
- `combinedOverlay`：设置为方块实体当前叠加层的整数，通常为 `OverlayTexture#NO_OVERLAY` 或 655,360。

注册 BER

为了注册 BER，你必须在模组事件总线上订阅 `EntityRenderersEvent$RegisterRenderers` 事件并调用 `#registerBlockEntityRenderer`。

物品

与方块一样，物品是大多数模组的关键组成部分。方块构成了你周围的地形，而物品存在于物品栏中。

创建物品

基础物品

不需要特殊功能的基础物品（例如木棍或糖）不需要自定义类。你可以通过使用 `Item$Properties` 对象实例化 `Item` 类来创建物品。这个 `Item$Properties` 对象可以通过构造函数创建，并通过调用其方法进行自定义。例如：

| 方法 | 描述 |
|-------------------------------|--|
| <code>requiredFeatures</code> | 设置需要在 <code>CreativeModeTab</code> 中看到此物品所需的 <code>FeatureFlag</code> 。 |
| <code>durability</code> | 设置此物品的最大耐久值。如果大于 0，则会添加两个物品属性“ <code>damaged</code> ”和“ <code>damage</code> ”。 |
| <code>stacksTo</code> | 设置最大堆叠数量。你不能同时拥有可损耗和可堆叠的物品。 |
| <code>setNoRepair</code> | 使此物品无法修复，即使它是可损耗的。 |
| <code>craftRemainder</code> | 设置此物品的容器物品，类似于熔岩桶使用后返还空桶的方式。 |

以上方法是可链式调用的，意味着它们 `return this` 以便于连续调用。

高级物品

如上所述设置物品属性仅适用于简单物品。如果你想要更复杂的物品，你应该继承 `Item` 并重写其方法。

创造模式标签页

物品可以通过模组事件总线上的 `BuildCreativeModeTabContentsEvent` 添加到 `CreativeModeTab` 中。可以通过 `#accept` 添加物品，无需任何额外配置。

```
// 在 MOD 事件总线上注册
// 假设我们有 RegistryObject<Item> 和 RegistryObject<Block> 分别名为 ITEM 和 BLOCK
@SubscribeEvent
public void buildContents(BuildCreativeModeTabContentsEvent event) {
    // 添加到材料标签页
    if (event.getTabKey() == CreativeModeTabs.INGREDIENTS) {
        event.accept(ITEM);
        event.accept(BLOCK); // 接受 ItemLike, 假设方块已注册物品
    }
}
```

你还可以通过 `FeatureFlagSet` 中的 `FeatureFlag` 或一个布尔值（确定玩家是否有权限查看 `Operator` 创造标签页）来启用或禁用物品的添加。

自定义创造模式标签页

自定义 `CreativeModeTab` 必须注册。构建器可以通过 `CreativeModeTab#builder` 创建。标签页可以设置标题、图标、默认物品以及其他一些属性。此外，Forge 还提供了额外的方法来来自定义标签页的图片、标签和槽位颜色、标签页的排序位置等。

```
// 假设我们有一个名为 REGISTRAR 的 DeferredRegister<CreativeModeTab>
// 假设我们有 RegistryObject<Item> 和 RegistryObject<Block> 分别名为 ITEM 和 BLOCK
public static final RegistryObject<CreativeModeTab> EXAMPLE_TAB = REGISTRAR.register("example", () -> CreativeModeTab.builder()
    // 设置标签页的显示名称
    .title(Component.translatable("item_group." + MOD_ID + ".example"))
    // 设置创造模式标签页的图标
    .icon(() -> new ItemStack(ITEM.get()))
    // 向标签页添加默认物品
    .displayItems((params, output) -> {
        output.accept(ITEM.get());
        output.accept(BLOCK.get());
    })
    .build()
);
```

注册物品

物品必须注册才能正常使用。

BlockEntityWithoutLevelRenderer

BlockEntityWithoutLevelRenderer (BEWLR) 是一种处理物品动态渲染的方法。该系统比旧的 ItemStack 系统简单得多，旧系统需要 BlockEntity，并且不允许访问 ItemStack。

使用 BlockEntityWithoutLevelRenderer

BlockEntityWithoutLevelRenderer 允许你使用 `public void renderByItem(ItemStack itemStack, ItemDisplayContext ctx, PoseStack poseStack, MultiBufferSource bufferSource, int combinedLight, int combinedOverlay)` 来渲染物品。

要使用 BEWLR，Item 必须首先满足其模型对 `BakedModel#isCustomRenderer` 返回 `true` 的条件。如果没有，它将使用默认的 `ItemRenderer#getBlockEntityRenderer`。一旦返回 `true`，该物品的 BEWLR 将被用于渲染。

!!! note 如果 `Block#getRenderShape` 设置为 `RenderShape#ENTITYBLOCK_ANIMATED`，Block 也会使用 BEWLR 进行渲染。

要为物品设置 BEWLR，必须在 `Item#initializeClient` 中消费 `IClientItemExtensions` 的匿名实例。在匿名实例中，应重写 `IClientItemExtensions#getCustomRenderer` 以返回你的 BEWLR 实例：

```
// 在你的物品类中
@Override
public void initializeClient(Consumer<IClientItemExtensions> consumer) {
    consumer.accept(new IClientItemExtensions() {

        @Override
        public BlockEntityWithoutLevelRenderer getCustomRenderer() {
            return myBEWLRInstance;
        }
    });
}
```

!!! important 每个模组应该只有一个自定义 BEWLR 实例。

就是这样，使用 BEWLR 不需要其他额外设置。

网络

服务器和客户端之间的通信是成功模组实现的基石。

网络通信有两个主要目标：

1. 确保客户端视图与服务器视图“同步”
 - 坐标 (X, Y, Z) 处的花朵刚刚生长
2. 为客户端提供一种方式，告知服务器玩家状态发生了变化
 - 玩家按下了某个键

实现这些目标的最常见方式是在客户端和服务器之间传递消息。这些消息通常是有结构的，包含特定排列的数据，以便于发送和接收。

Forge 提供了多种通信技术，大多构建在 netty 之上。

对于新模组来说，最简单的方式是 SimpleImpl，它抽象了 netty 系统的大部分复杂性。它使用消息和处理器的风格系统。

SimpleImpl

SimpleImpl 是围绕 SimpleChannel 类的数据包系统的名称。使用此系统是在客户端和服务器之间发送自定义数据的最简单方式。

入门

首先，你需要创建你的 SimpleChannel 对象。我们建议在一个单独的类中执行此操作，可能类似于 ModidPacketHandler。将你的 SimpleChannel 创建为此类中的静态字段，如下所示：

```
private static final String PROTOCOL_VERSION = "1";
public static final SimpleChannel INSTANCE = NetworkRegistry.newSimpleChannel(
    new ResourceLocation("mymodid", "main"),
    () -> PROTOCOL_VERSION,
    PROTOCOL_VERSION::equals,
    PROTOCOL_VERSION::equals
);
```

第一个参数是通道的名称。第二个参数是返回当前网络协议版本的 Supplier<String>。第三个和第四个参数分别是 Predicate<String>，用于检查传入连接协议版本是否与客户端或服务器网络兼容。

在这里，我们直接与 PROTOCOL_VERSION 字段进行比较，这意味着客户端和服务器的 PROTOCOL_VERSION 必须始终匹配，否则 FML 将拒绝登录。

版本检查器

如果你的模组不需要另一端具有特定的网络通道，或者根本不要求另一端是 Forge 实例，你应注意正确定义版本兼容性检查器 (Predicate<String> 参数)，以处理版本检查器可能接收到的额外“元版本”（在 NetworkRegistry 中定义）。这些包括：

- ABSENT —如果另一端缺少此通道。注意，在这种情况下，另一端仍然是 Forge 端点，并且可能具有其他模组。
- ACCEPTVANILLA —如果端点是原版（或非 Forge）端点。

两者都返回 false 意味着此通道必须存在于另一端。如果你只是复制上面的代码，这就是它所做的。注意，这些值也在列表 ping 兼容性检查期间使用，该检查负责在多人服务器选择屏幕中显示绿色勾选或红色叉号。

注册数据包

接下来，我们必须声明我们想要发送和接收的消息类型。这通过 INSTANCE#registerMessage 完成，它接受 5 个参数：

- 第一个参数是数据包的判别符 (discriminator)。这是每个通道唯一的数据包 ID。我们建议使用局部变量来保存 ID，然后使用 id+ 调用 registerMessage。这将保证 100% 唯一的 ID。
- 第二个参数是实际的数据包类 MSG。
- 第三个参数是 BiConsumer<MSG, FriendlyByteBuf>，负责将消息编码到提供的 FriendlyByteBuf 中。
- 第四个参数是 Function<FriendlyByteBuf, MSG>，负责从提供的 FriendlyByteBuf 解码消息。
- 最后一个参数是 BiConsumer<MSG, Supplier<NetworkEvent.Context>>，负责处理消息本身。

最后三个参数可以是 Java 中静态方法或实例方法的方法引用。记住，实例方法 MSG#encode(FriendlyByteBuf) 仍然满足 BiConsumer<MSG, FriendlyByteBuf>; MSG 只是成为隐式的第一个参数。

处理数据包

在数据包处理器中有几点需要强调。数据包处理器可以访问消息对象和网络上下文。上下文允许访问发送数据包的玩家（如果在服务器上），以及一种排队线程安全工作的方法。

```
public static void handle(MyMessage msg, Supplier<NetworkEvent.Context> ctx) {
    ctx.get().enqueueWork(() -> {
        // 需要线程安全的工作（大多数工作）
        ServerPlayer sender = ctx.get().getSender(); // 发送此数据包的客户端
        // 执行操作
    });
    ctx.get().setPacketHandled(true);
}
```

从服务器发送到客户端的数据包应在另一个类中处理，并通过 DistExecutor#unsafeRunWhenOn 包装。

```
// 在数据包类中
public static void handle(MyClientMessage msg, Supplier<NetworkEvent.Context> ctx) {
    ctx.get().enqueueWork(() -> {
        DistExecutor.unsafeRunWhenOn(Dist.CLIENT, () -> () -> ClientPacketHandlerClass.handlePacket(msg, ctx))
    });
    ctx.get().setPacketHandled(true);
}
```



```

}

// 在 ClientPacketHandlerClass 中
public static void handlePacket(MyClientMessage msg, Supplier<NetworkEvent.Context> ctx) {
    // 执行操作
}

```

注意 #setPacketHandled 的存在，它用于告知网络系统数据包已成功处理完成。

!!! warning 从 Minecraft 1.8 开始，数据包默认在网络线程上处理。

这意味着你的处理器**不能**直接与大多数游戏对象交互。Forge 通过提供的 `NetworkEvent\$Context` 提供了一种便捷方式，让你

!!! warning 在服务器上处理数据包时要保持防御性。客户端可能会尝试通过发送意外数据来利用数据包处理。

一个常见的问题是**任意区块生成**漏洞。这通常发生在服务器信任客户端发送的方块位置来访问方块和方块实体时。当访问未加载

为避免此问题，一个通用的经验法则是仅在 `Level#hasChunkAt` 为 `true` 时访问方块和方块实体。

发送数据包

发送到服务器

只有一种方式可以向服务器发送数据包。这是因为客户端一次只能连接一个服务器。为此，我们必须再次使用之前定义的 SimpleChannel。只需调用 INSTANCE.sendToServer(new MyMessage())。如果存在该类型的处理器，消息将被发送到该处理器。

发送到客户端

可以使用 SimpleChannel 直接将数据包发送到客户端：HANDLER.sendTo(new MyClientMessage(), serverPlayer.connection.getConnection(), NetworkDirection.PLAY_TO_CLIENT)。然而，这可能相当不方便。Forge 提供了一些便利函数：

```

// 发送给一个玩家
INSTANCE.send(PacketDistributor.PLAYER.with(serverPlayer), new MyMessage());

// 发送给所有追踪此区块的玩家
INSTANCE.send(PacketDistributor.TRACKING_CHUNK.with(levelChunk), new MyMessage());

// 发送给所有连接的玩家
INSTANCE.send(PacketDistributor.ALL.noArg(), new MyMessage());

```

还有其他可用的 PacketDistributor 类型；请查看 PacketDistributor 类的文档以获取更多详细信息。

实体

除了常规的网络消息之外，还有其他各种系统可用于处理实体数据的同步。

生成数据

通常，模组实体的生成由 Forge 单独处理。

!!! note 这意味着简单地继承原版实体类可能不会继承其所有行为。你可能需要自己实现某些原版行为。

你可以通过实现以下接口向 Forge 发送的生成数据包中添加额外数据。

IEntityAdditionalSpawnData

如果你的实体具有客户端需要但不会随时间变化的数据，则可以使用此接口将其添加到实体生成数据包中。#writeSpawnData 和 #readSpawnData 控制数据应如何编码/解码到网络缓冲区。

动态数据

数据参数 (Data Parameters)

这是从服务器到客户端同步实体数据的主要原版系统。因此，有许多原版示例可供参考。

首先，你需要为想要保持同步的数据创建一个 `EntityDataAccessor<T>`。这应存储为实体类中的 `static final` 字段，通过调用 `SynchedEntityData#defineId` 并传入实体类和数据类型的序列化器获得。可用的序列化器实现可以在 `EntityDataSerializers` 类中找到（作为静态常量）。

!!! warning 你**只**应为你自己的实体创建数据参数，并且在该实体的类中。向你无法控制的实体添加参数可能导致用于通过网络发送数据的 ID 不同步，从而造成难以调试的崩溃。

然后，重写 `Entity#defineSynchedData` 并为每个数据参数调用 `this.entityData.define(...)`，传入参数和初始值。记得始终首先调用 `super` 方法！

然后，你可以通过实体的 `entityData` 实例获取和设置这些值。所做的更改将自动同步到客户端。

粒子

粒子是游戏中用于提升沉浸感的特效。

粒子系统分为四个主要组件：

| 类 | 端 | 描述 |
|-------------------------------|--------|---|
| <code>ParticleType</code> | 双端 | 粒子类型的注册对象，用于引用粒子 |
| <code>ParticleOptions</code> | 双端 | 用于从网络或命令同步粒子数据的数据持有者 |
| <code>ParticleProvider</code> | CLIENT | 工厂，由 <code>ParticleType</code> 注册，用于从 <code>ParticleOptions</code> 构造 <code>Particle</code> |
| <code>Particle</code> | CLIENT | 在客户端渲染的可渲染逻辑 |

ParticleType

必须注册。接受两个参数：`overrideLimiter`（是否无视距离渲染）和 `ParticleOptions$Deserializer`。简单粒子可继承 `SimpleParticleType`。

ParticleOptions

包含三种方法：`getType`（获取 `ParticleType`）、`writeToNetwork`（写入网络缓冲区）、`writeToString`（写入字符串）。`SimpleParticleType` 同时实现了 `ParticleType` 和 `ParticleOptions`。

Particle

实现 `render` 和 `getRenderType`。常用子类 `TextureSheetParticle`。- `ParticleRenderType`: `TERRAIN_SHEET` / `PARTICLE_SHEET_OPAQUE` / `PARTICLE_SHEET_TRANSLUCENT` / `PARTICLE_SHEET_LIT` / `CUSTOM` / `NO_RENDER`

ParticleProvider

在 `RegisterParticleProvidersEvent`（模组事件总线）中注册。- `#registerSpecial` —注册自定义 `ParticleProvider` - `#registerSpriteSet` —注册带精灵表的粒子（需在 `assets/<modid>/particles/<name>.json` 中定义纹理列表）

```
// assets/mymod/particles/my_particle.json
{ "textures": [ "mymod:particle_texture", "mymod:particle_texture2" ] }
```

生成粒子

- `ClientLevel`: `#addParticle` / `#addAlwaysVisibleParticle`
- `ServerLevel`: `#sendParticles`（发送数据包到客户端）

声音

术语: - **Sound Event** —触发声音效果的事件, 如 `minecraft:block.anvil.hit` - **Sound Category** —声音类别 (player、block、master 等), 对应设置中的滑块 - **Sound File** —实际的.ogg 音频文件

`sounds.json` 位于 `assets/<namespace>/sounds.json`, 定义声音事件及其关联的音频文件。

```
{
  "open_chest": {
    "subtitle": "mymod.subtitle.open_chest",
    "sounds": [ "mymod:open_chest_sound_file" ]
  },
  "epic_music": {
    "sounds": [ { "name": "mymod:music/epic_music", "stream": true } ]
  }
}
```

- 长音频 (背景音乐、唱片) 应使用 `stream: true` 避免加载到内存
- 音频文件位于 `assets/<namespace>/sounds/<path>.ogg`
- `sounds.json` 可通过数据生成生成

SoundEvent

必须在代码中创建并注册 `SoundEvent`, 作为在服务器端播放声音的引用。

播放声音方法

- `Level#playSound(Player, x, y, z, SoundEvent, source, vol, pitch)` —服务端: 播放给附近除传入玩家外的所有人; 客户端: 仅播放给传入的客户端玩家
- `Level#playLocalSound(x, y, z, SoundEvent, source, vol, pitch, distDelay)` —**仅客户端**, 用于自定义数据包或客户端特效
- `Entity#playSound(SoundEvent, vol, pitch)` —服务端: 在实体位置播放给所有人。客户端: 无操作
- `Player#playSound(SoundEvent, vol, pitch)` —服务端: 播放给除自己外的附近玩家。客户端: 见 `LocalPlayer`

能力系统

Capability (能力) 系统允许以动态灵活的方式暴露功能, 无需直接实现大量接口。

Forge 提供的能力: - `IItemHandler` —物品栏处理 (替代旧的 `Container/WorldlyContainer`) - `IFluidHandler` —流体处理 - `IEnergyStorage` —能量处理 (基于 `RedstoneFlux API`)

使用能力

通过 `#getCapability(cap, direction)` 查询。返回 `LazyOptional<T>`。- `cap`: 能力实例, 通过 `CapabilityManager.get(new CapabilityToken<>())` 获取 - `direction`: 方向, `null` 表示无方向要求 - 使用前检查 `Capability#isRegistered`

暴露能力

1. 创建能力接口的实例 (如 `ItemStackHandler`)
2. 重写 `#getCapability`, 比较能力实例并返回对应 `LazyOptional`
3. 在 `#invalidateCaps` 或 `AttachCapabilitiesEvent#addListener` 中失效 `LazyOptional`

```
@Override
public <T> LazyOptional<T> getCapability(Capability<T> cap, Direction side) {
    if (cap == ForgeCapabilities.ITEM_HANDLER) {
        return inventoryHandlerLazyOptional.cast();
    }
    return super.getCapability(cap, side);
}
```

附加能力 (给非自有对象)

使用 `AttachCapabilitiesEvent` (在 Forge 总线上触发) 为 `Entity`、`BlockEntity`、`ItemStack`、`Level`、`LevelChunk` 附加能力。实现 `ICapabilitySerializable` 以支持持久化。

注册自定义能力

- `RegisterCapabilitiesEvent` (模组事件总线): `event.register(IExampleCapability.class)`
- `@AutoRegisterCapability`: 注解在能力接口上

数据同步

能力数据默认不发送到客户端。需要通过自定义网络数据包实现同步。

玩家死亡持久化

在 `PlayerEvent$Clone` 中手动复制数据, 通过 `#isWasDeath` 区分死亡重生和从未地返回。

持久化数据 (SavedData)

`SavedData` (SD) 系统是 `Level Capabilities` 的替代方案, 用于为每个维度附加持久化数据。

声明

继承 `SavedData`, 实现两个方法: - `#save(tag)` — 将数据写入 NBT - `#setDirty()` — 数据变更后调用, 否则 `#save` 不会被调用

附加到维度

通过 `DimensionDataStorage#computeIfAbsent(loader, factory, name)` 创建/加载 SD: - `loader`: 从 NBT 加载数据的函数 - `factory`: 创建新实例的供应者 - `name`: `.dat` 文件名 (不包含扩展名)

```
// 在 Nether (DIM-1) 中创建 ./<level_folder>/DIM-1/data/example.dat  
netherDataStorage.computeIfAbsent(this::load, this::create, "example");
```

跨维度持久化

将 SD 附加到主世界 (Overworld), 通过 `MinecraftServer#overworld` 获取, 因为主世界是唯一永远不会完全卸载的维度。

编解码器 (Codec)

Codec 是 Mojang 的 `DataFixerUpper` 提供的序列化工具, 用于在 JSON (`JsonElement`) 和 NBT (`Tag`) 等格式之间转换对象。

使用 Codec

- `Codec#encodeStart(ops, obj)` — 编码
- `Codec#parse(ops, json)` — 解码
- `DynamicOps`: `JsonOps.INSTANCE` (标准 JSON)、`JsonOps.COMPRESSED` (压缩)、`NbtOps.INSTANCE` (NBT)
- 返回 `DataResult`, 使用 `.resultOrPartial(err -> {})` 获取结果

现有 Codec

- 基础类型: `Codec.INT`, `Codec.STRING`, `Codec.BOOL`, `Codec.FLOAT` 等
- 原版: `ResourceLocation#CODEC`, `CompoundTag#CODEC`
- 注册表: `Registry#byNameCodec`, `IForgeRegistry#getCodec`

创建 Codec

Record (记录类型):

```
public static final Codec<SomeObject> CODEC = RecordCodecBuilder.create(instance ->
    instance.group(
        Codec.STRING.fieldOf("s").forGetter(SomeObject::s),
        Codec.INT.optionalFieldOf("i", 0).forGetter(SomeObject::i)
    ).apply(instance, SomeObject::new)
);
```

转换器: - #xmap(A→B, B→A) —双向完全等价 - #comapFlatMap(A→DataResult, B→A) —解码有约束 - #flatComapMap(A→B, B→DataResult<A>) —编码有约束 - #flatMap(A→DataResult, B→DataResult<A>) —双向有约束 - #intRange(min, max) —整数范围约束

其他: - #orElse(default) —编解码失败时使用默认值 - #unit(supplier) —仅提供代码内值, 不编解码 - #listOf() —列表 - #unboundedMap(keyCodec, valueCodec) —映射 - #pair(codec1, codec2) —对 - #either(codec1, codec2) —二选一 - #dispatch(typeGetter, codecGetter) —分发, 根据 type 字段选择子 codec

菜单

菜单是图形用户界面 (GUI) 的一种后端类型; 它们处理与某个表示数据持有者交互的逻辑。菜单本身不是数据持有者, 而是允许用户间接修改内部数据持有者状态的视图。因此, 数据持有者不应与任何菜单直接耦合, 而是应传入数据引用以进行调用和修改。

MenuType

菜单是动态创建和移除的, 因此它们本身不是注册表对象。相应地, 会注册另一个工厂对象来轻松创建和引用菜单的类型。对于菜单, 这些被称为 MenuType。

MenuType 必须注册。

MenuSupplier

MenuType 通过向其构造函数传入 MenuSupplier 和 FeatureFlagSet 来创建。MenuSupplier 是一个函数, 接受容器的 ID 和查看菜单的玩家的物品栏, 并返回一个新创建的 AbstractContainerMenu。

```
// 对于某个 DeferredRegister<MenuType<?>> REGISTER
public static final RegistryObject<MenuType<MyMenu>> MY_MENU = REGISTER.register("my_menu", () -> new MenuType(MyMenu::new,
    FeatureFlags.DEFAULT_FLAGS));

// 在 MyMenu 中, 一个 AbstractContainerMenu 的子类
public MyMenu(int containerId, Inventory playerInv) {
    super(MY_MENU.get(), containerId);
    // ...
}
```

!!! note 容器标识符对于单个玩家是唯一的。这意味着在两个不同玩家上的相同容器 ID 将代表两个不同的菜单, 即使他们正在查看相同的数据持有者。

MenuSupplier 通常负责在客户端上创建菜单, 使用虚拟数据引用用来存储和与来自服务器数据持有者的同步信息进行交互。

IContainerFactory

如果客户端上需要额外信息 (例如数据持有者在世界中的位置), 则可以使用子类 IContainerFactory。除了容器 ID 和玩家物品栏之外, 它还提供了一个 FriendlyByteBuf, 可以存储从服务器发送的额外信息。可以通过 IForgeMenuType#create 使用 IContainerFactory 创建 MenuType。

```
// 对于某个 DeferredRegister<MenuType<?>> REGISTER
public static final RegistryObject<MenuType<MyMenuExtra>> MY_MENU_EXTRA = REGISTER.register("my_menu_extra", () ->
    IForgeMenuType.create(MyMenu::new));

// 在 MyMenuExtra 中, 一个 AbstractContainerMenu 的子类
public MyMenuExtra(int containerId, Inventory playerInv, FriendlyByteBuf extraData) {
    super(MY_MENU_EXTRA.get(), containerId);
    // 从缓冲区存储额外数据
}
```

```
// ...
}
```

AbstractContainerMenu

所有菜单都继承自 AbstractContainerMenu。菜单接受两个参数：MenuType（表示菜单本身的类型）和容器 ID（表示当前访问者的菜单唯一标识符）。

!!! important 玩家一次最多只能打开 100 个唯一的菜单。

每个菜单应包含两个构造函数：一个用于在服务器上初始化菜单，另一个用于在客户端上初始化菜单。用于在客户端上初始化菜单的构造函数是提供给 MenuType 的那个。服务器菜单构造函数包含的任何字段都应在客户端菜单构造函数中具有一些默认值。

```
// 客户端菜单构造函数
public MyMenu(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory);
}

// 服务器菜单构造函数
public MyMenu(int containerId, Inventory playerInventory) {
    // ...
}
```

每个菜单实现必须实现两个方法：#stillValid 和 #quickMoveStack。

#stillValid 和 ContainerLevelAccess

#stillValid 确定菜单是否应对给定玩家保持打开状态。通常这指向静态的 #stillValid，它接受一个 ContainerLevelAccess、玩家和此菜单所附加的 Block。客户端菜单必须始终返回 true，而静态 #stillValid 默认也是如此。此实现检查玩家是否在数据存储对象所在位置的 8 个方块范围内。

ContainerLevelAccess 在封闭的作用域内提供当前世界和方块的位置。在服务器上构建菜单时，可以通过调用 ContainerLevelAccess#create 创建新的访问对象。客户端菜单构造函数可以传入 ContainerLevelAccess#NULL，它不会执行任何操作。

```
// 客户端菜单构造函数
public MyMenuAccess(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory, ContainerLevelAccess.NULL);
}

// 服务器菜单构造函数
public MyMenuAccess(int containerId, Inventory playerInventory, ContainerLevelAccess access) {
    // ...
}

// 假设此菜单附加到 RegistryObject<Block> MY_BLOCK
@Override
public boolean stillValid(Player player) {
    return AbstractContainerMenu.stillValid(this.access, player, MY_BLOCK.get());
}
```

数据同步

某些数据需要在服务器和客户端上都存在以显示给玩家。为此，菜单实现了一个基本的数据同步层，每当当前数据与上次同步到客户端的数据不匹配时进行同步。对于玩家，每 tick 检查一次。

Minecraft 默认支持两种形式的数据同步：通过 Slot 的 ItemStack 和通过 DataSlot 的整数。Slot 和 DataSlot 是持有对数据存储引用的视图，玩家可以在屏幕中修改它们（假设操作有效）。这些可以在构造函数中通过 #addSlot 和 #addDataSlot 添加到菜单中。

!!! note 由于 Slot 使用的 Container 已被 Forge 废弃，推荐使用 IItemHandler 能力，因此其余的说明将围绕使用能力变体：SlotItemHandler。

SlotItemHandler 包含四个参数：表示栈所在物品栏的 IItemHandler、此槽位具体代表的栈的索引，以及槽位左上角在屏幕上相对于 AbstractContainerScreen#leftPos 和 #topPos 渲染的 x 和 y 位置。客户端菜单构造函数应始终提供相同大小的空物品栏实例。

在大多数情况下，先添加菜单包含的所有槽位，然后是玩家物品栏，最后以玩家快捷栏结尾。要从菜单访问任何单个 Slot，必须根据添加槽位的顺序计算索引。

DataSlot 是一个抽象类，应实现 getter 和 setter 来引用存储在数据存储对象中的数据。客户端菜单构造函数应始终通过 DataSlot#standalone 提供一个新的实例。

这些与槽位一起，应在每次初始化新菜单时重新创建。

!!! warning 尽管 DataSlot 存储一个整数，但由于其通过网络发送值的方式，实际上限制为 short (-32768 到 32767)。整数的高 16 位被忽略。

```
// 假设我们有一个大小为 5 的数据对象物品栏
// 假设我们在每次初始化服务器菜单时构造了一个 DataSlot

// 客户端菜单构造函数
public MyMenuAccess(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory, new ItemStackHandler(5), DataSlot.standalone());
}

// 服务器菜单构造函数
public MyMenuAccess(int containerId, Inventory playerInventory, IItemHandler dataInventory, DataSlot dataSingle) {
    // 检查数据物品栏大小是否为某个固定值
    // 然后，为数据物品栏添加槽位
    this.addSlot(new SlotItemHandler(dataInventory, /*...*/));

    // 为玩家物品栏添加槽位
    this.addSlot(new Slot(playerInventory, /*...*/));

    // 为处理的整数添加数据槽位
    this.addDataSlot(dataSingle);

    // ...
}
```

ContainerData 如果需要将多个整数同步到客户端，可以使用 ContainerData 来引用这些整数。此接口作为一个索引查找表，每个索引代表一个不同的整数。ContainerData 也可以在数据对象本身中构造，如果 ContainerData 通过 #addDataSlots 添加到菜单中。该方法为接口指定的数据量为每个数据创建一个新的 DataSlot。客户端菜单构造函数应始终通过 SimpleContainerData 提供一个新的实例。

```
// 假设我们有一个大小为 3 的 ContainerData

// 客户端菜单构造函数
public MyMenuAccess(int containerId, Inventory playerInventory) {
    this(containerId, playerInventory, new SimpleContainerData(3));
}

// 服务器菜单构造函数
public MyMenuAccess(int containerId, Inventory playerInventory, ContainerData dataMultiple) {
    // 检查 ContainerData 大小是否为某个固定值
    checkContainerDataCount(dataMultiple, 3);

    // 为处理的整数添加数据槽位
    this.addDataSlots(dataMultiple);

    // ...
}
```

!!! warning 由于 ContainerData 委托给 DataSlot，因此也限制为 short (-32768 到 32767)。

#quickMoveStack #quickMoveStack 是每个菜单必须实现的第二个方法。每当堆叠的物品被 Shift 点击（即快速移动）出其当前槽位时，就会调用此方法，直到堆叠的物品完全移出其之前的槽位或没有其他地方可去为止。该方法返回被快速移动的槽位中堆叠物品的副本。

堆叠的物品通常使用 #moveItemStackTo 在槽位之间移动，该方法将堆叠物品移动到第一个可用的槽位。它接受要移动的堆叠物品、第一个槽位索引（包含）、最后一个槽位索引（不包含）以及是否从第一个到最后一个检查槽位（false）或从最后一个到第一个检查槽位（true）。

在 Minecraft 的实现中，此方法的逻辑相当一致：

```
// 假设我们有一个大小为 5 的数据物品栏
// 该物品栏有 4 个输入槽（索引 1-4），输出到结果槽（索引 0）
// 我们还有 27 个玩家物品栏槽位和 9 个快捷栏槽位
// 因此，实际槽位的索引如下：
//   - 数据物品栏：结果（0），输入（1-4）
//   - 玩家物品栏（5-31）
```

```

// - 玩家快捷栏 (32-40)
@Override
public ItemStack quickMoveStack(Player player, int quickMovedSlotIndex) {
    // 快速移动的槽位堆叠
    ItemStack quickMovedStack = ItemStack.EMPTY;
    // 快速移动的槽位
    Slot quickMovedSlot = this.slots.get(quickMovedSlotIndex)

    // 如果槽位在有效范围内且不为空
    if (quickMovedSlot != null && quickMovedSlot.hasItem()) {
        // 获取要移动的原始堆叠
        ItemStack rawStack = quickMovedSlot.getItem();
        // 将槽位堆叠设置为原始堆叠的副本
        quickMovedStack = rawStack.copy();

        // 如果快速移动是在数据物品栏结果槽位上执行的
        if (quickMovedSlotIndex == 0) {
            // 尝试将结果槽移动到玩家物品栏/快捷栏
            if (!this.moveItemStackTo(rawStack, 5, 41, true)) {
                return ItemStack.EMPTY;
            }
            slot.onQuickCraft(rawStack, quickMovedStack);
        }
        // 否则如果快速移动是在玩家物品栏或快捷栏槽位上执行的
        else if (quickMovedSlotIndex >= 5 && quickMovedSlotIndex < 41) {
            // 尝试将物品栏/快捷栏槽位移入数据物品栏输入槽
            if (!this.moveItemStackTo(rawStack, 1, 5, false)) {
                // 如果在玩家物品栏槽位中且无法移动, 尝试移动到快捷栏
                if (quickMovedSlotIndex < 32) {
                    if (!this.moveItemStackTo(rawStack, 32, 41, false)) {
                        return ItemStack.EMPTY;
                    }
                }
                // 否则尝试将快捷栏移动到玩家物品栏槽位
                else if (!this.moveItemStackTo(rawStack, 5, 32, false)) {
                    return ItemStack.EMPTY;
                }
            }
        }
        // 否则如果快速移动是在数据物品栏输入槽上执行的, 尝试移动到玩家物品栏/快捷栏
        else if (!this.moveItemStackTo(rawStack, 5, 41, false)) {
            return ItemStack.EMPTY;
        }

        if (rawStack.isEmpty()) {
            quickMovedSlot.set(ItemStack.EMPTY);
        } else {
            quickMovedSlot.setChanged();
        }

        if (rawStack.getCount() == quickMovedStack.getCount()) {
            return ItemStack.EMPTY;
        }
        quickMovedSlot.onTake(player, rawStack);
    }

    return quickMovedStack;
}

```

打开菜单

一旦菜单类型已注册、菜单本身已完成并且已附加了屏幕，玩家就可以打开菜单。可以通过在逻辑服务器上调用 `NetworkHooks#openScreen` 来打开菜单。该方法接受打开菜单的玩家、服务端菜单的 `MenuProvider`，以及可选的 `FriendlyByteBuf`（如果需要将额外数据同步到客户端）。

!!! note 只有在使用 `IContainerFactory` 创建菜单类型时，才应使用带有 `FriendlyByteBuf` 参数的 `NetworkHooks#openScreen`。

MenuProvider `MenuProvider` 是一个包含两个方法的接口：`#createMenu`（创建菜单的服务器实例）和 `#getDisplayName`（返回包含菜单标题的组件，传递给屏幕）。`#createMenu` 方法包含三个参数：菜单的容器 ID、打开菜单的玩家的物品栏以及打开菜单的玩家。

可以使用 SimpleMenuProvider 轻松创建 MenuProvider，它接受一个创建服务器菜单的方法引用和菜单的标题。

```
// 在某些实现中
NetworkHooks.openScreen(serverPlayer, new SimpleMenuProvider(
    (containerId, playerInventory, player) -> new MyMenu(containerId, playerInventory),
    Component.translatable("menu.title.examplemod.mymenu")
));
```

常见实现

菜单通常在某种玩家交互时打开（例如，右键点击方块或实体时）。

方块实现 方块通常通过重写 BlockBehaviour#use 来实现菜单。如果在逻辑客户端上，交互返回 InteractionResult#SUCCESS。否则，它打开菜单并返回 InteractionResult#CONSUME。

MenuProvider 应通过重写 BlockBehaviour#getMenuProvider 来实现。原版方法使用它来在旁观者模式下查看菜单。

```
// 在某些 Block 子类中
@Override
public MenuProvider getMenuProvider(BlockState state, Level level, BlockPos pos) {
    return new SimpleMenuProvider(/* ... */);
}

@Override
public InteractionResult use(BlockState state, Level level, BlockPos pos, Player player, InteractionHand hand, BlockHitResult result) {
    if (!level.isClientSide && player instanceof ServerPlayer serverPlayer) {
        NetworkHooks.openScreen(serverPlayer, state.getMenuProvider(level, pos));
    }
    return InteractionResult.sidedSuccess(level.isClientSide);
}
```

!!! note 这是实现此逻辑的最简单方式，但不是唯一方式。如果你希望方块仅在特定条件下打开菜单，则需要事先将某些数据同步到客户端，以便在条件不满足时返回 InteractionResult#PASS 或 #FAIL。

生物实现 生物通常通过重写 Mob#mobInteract 来实现菜单。这与方块实现类似，唯一的区别在于 Mob 本身应实现 MenuProvider 以支持旁观者模式查看。

```
public class MyMob extends Mob implements MenuProvider {
    // ...

    @Override
    public InteractionResult mobInteract(Player player, InteractionHand hand) {
        if (!this.level.isClientSide && player instanceof ServerPlayer serverPlayer) {
            NetworkHooks.openScreen(serverPlayer, this);
        }
        return InteractionResult.sidedSuccess(this.level.isClientSide);
    }
}
```

!!! note 同样，这是实现此逻辑的最简单方式，但不是唯一方式。

屏幕

屏幕通常是 Minecraft 中所有图形用户界面（GUI）的基础：接受用户输入、在服务器上验证，并将结果操作同步回客户端。它们可以与菜单结合，为类似物品栏的视图创建通信网络，也可以是独立的，模组开发者可以通过自己的网络实现来处理。

屏幕由许多部分组成，因此很难完全理解 Minecraft 中“屏幕”的实际含义。因此，本文档将在讨论屏幕本身之前，先介绍屏幕的每个组成部分及其应用方式。

相对坐标

每当渲染任何内容时，都需要有一个标识符来指定它将在哪里出现。通过众多的抽象层，Minecraft 的大多数渲染调用都接受坐标平面中的 x、y 和 z 值。X 值从左到右递增，Y 值从上到下递增，Z 值从远到近递增。然而，坐标并非固定在一个指定的范围内。它们可以根据屏幕的大小和在选项中指定的缩放比例而变化。因此，必须格外小心，以确保渲染坐标的值能够适当地缩放到可变的屏幕大小。

关于如何相对化坐标的信息将在屏幕部分中介绍。

!!! important 如果你选择使用固定坐标或不正确地缩放屏幕，渲染的对象可能会看起来很奇怪或位置错误。检查坐标相对化是否正确的一个简单方法是点击视频设置中的“GUI 缩放”按钮。在确定 GUI 应渲染的缩放比例时，该值用作显示宽度和高度的除数。

GuiGraphics

Minecraft 渲染的任何 GUI 通常都使用 GuiGraphics 完成。GuiGraphics 是几乎所有渲染方法的第一个参数；它包含渲染常用对象的基本方法。这些方法分为五类：彩色矩形、字符串、纹理、物品和提示框。还有一个用于渲染组件片段的方法（#enableScissor / #disableScissor）。GuiGraphics 还公开了 PoseStack，用于应用必要的变换以在正确的位置渲染组件。此外，颜色采用 ARGB 格式。

彩色矩形

彩色矩形通过位置颜色着色器绘制。可以绘制三种类型的彩色矩形。

首先是彩色水平线和垂直线（宽度为 1 像素），分别是 #hLine 和 #vLine。#hLine 接受两个 x 坐标（定义左右边界，包含）、顶部的 y 坐标和颜色。#vLine 接受左侧 x 坐标、两个 y 坐标（定义上下边界，包含）和颜色。

其次是 #fill 方法，它绘制一个矩形到屏幕上。线条方法内部调用此方法。它接受左侧 x 坐标、顶部 y 坐标、右侧 x 坐标、底部 y 坐标和颜色。

最后是 #fillGradient 方法，它绘制一个带有垂直渐变的矩形。它接受右侧 x 坐标、底部 y 坐标、左侧 x 坐标、顶部 y 坐标、z 坐标以及底部和顶部的颜色。

字符串

字符串通过其 Font 绘制，通常使用它们自己的着色器来支持普通、透视和偏移模式。可以渲染两种对齐方式的字符串，每种都有背景阴影：左对齐字符串（#drawString）和居中对齐字符串（#drawCenteredString）。两者都接受渲染字符串所用的字体、要绘制的字符串、分别表示字符串左侧或中心的 x 坐标、顶部 y 坐标和颜色。

!!! note 字符串通常应以 Component 的形式传入，因为它们可以处理各种使用场景，包括该方法的其他两个重载。

纹理

纹理通过 blitting（位块传输）绘制，因此方法名为 #blit，它复制图像的位并直接绘制到屏幕上。这些通过位置纹理着色器绘制。虽然有许多不同的 #blit 重载，但我们只讨论两个静态的 #blit。

第一个静态 #blit 接受六个整数，并假设正在渲染的纹理位于 256 x 256 的 PNG 文件中。它接受屏幕上的左侧 x 和顶部 y 坐标、PNG 内的左侧 x 和顶部 y 坐标，以及要渲染的图像的宽度和高度。

!!! note 必须指定 PNG 文件的大小，以便归一化坐标以获取相关的 UV 值。

第一个调用的静态 #blit 将其扩展为九个整数，仅假设图像位于 PNG 文件中。它接受屏幕上的左侧 x 和顶部 y 坐标、z 坐标（称为 blit 偏移）、PNG 内的左侧 x 和顶部 y 坐标、要渲染的图像的宽度和高度，以及 PNG 文件的宽度和高度。

Blit 偏移 渲染纹理时的 z 坐标通常设置为 blit 偏移。该偏移负责在查看屏幕时正确地对渲染进行分层。z 坐标较小的渲染在背景中渲染，反之，z 坐标较大的渲染在前景中渲染。可以通过 PoseStack 上的 #translate 直接设置 z 偏移。GuiGraphics 的某些方法内部应用了一些基本的偏移逻辑（例如物品渲染）。

!!! important 设置 blit 偏移后，必须在渲染对象后重置它。否则，屏幕中的其他对象可能会在不正确的层中渲染，导致图形问题。建议在平移之前推送当前 pose，然后在偏移处完成所有渲染后弹出。

Renderable

Renderable 本质上是可渲染的对象。这些包括屏幕、按钮、聊天框、列表等。Renderable 只有一个方法：#render。它接受用于将内容渲染到屏幕的 GuiGraphics、缩放到相对屏幕大小的鼠标 x 和 y 位置，以及 tick 增量（自上一帧以来经过的 tick 数）。

一些常见的可渲染对象是屏幕和“小部件”：通常在屏幕上渲染的可交互元素，例如 Button、其子类型 ImageButton，以及用于在屏幕上输入文本的 EditText。

GuiEventListener

Minecraft 中渲染的任何屏幕都实现了 `GuiEventListener`。`GuiEventListener` 负责处理用户与屏幕的交互。这些包括来自鼠标（移动、点击、释放、拖动、滚动、悬停）和键盘（按下、释放、输入）的输入。每个方法返回关联操作是否成功影响了屏幕。按钮、聊天框、列表等小部件也实现了此接口。

ContainerEventHandler

与 `GuiEventListener` 几乎同义的是其子类型：`ContainerEventHandler`。它们负责处理包含小部件的屏幕上的用户交互，管理当前聚焦的元素以及关联的交互如何应用。`ContainerEventHandler` 添加了三个额外功能：可交互子元素、拖拽和聚焦。

事件处理器持有子元素，用于确定元素的交互顺序。在鼠标事件处理器（不包括拖拽）期间，列表中鼠标悬停的第一个子元素会执行其逻辑。

使用鼠标拖拽元素，通过 `#mouseClicked` 和 `#mouseReleased` 实现，提供更精确的执行逻辑。

聚焦允许在事件执行期间（例如在键盘事件或鼠标拖拽期间）首先检查并处理特定的子元素。焦点通常通过 `#setFocused` 设置。此外，可以使用 `#nextFocusPath` 循环切换可交互子元素，根据传入的 `FocusNavigationEvent` 选择子元素。

!!! note 屏幕通过 `AbstractContainerEventHandler` 实现 `ContainerEventHandler`，它添加了拖拽和聚焦子元素的 setter 和 getter 逻辑。

NarratableEntry

`NarratableEntry` 是可以通过 Minecraft 的无障碍朗读功能进行语音描述的元素。每个元素可以根据悬停或选择的内容提供不同的叙述，通常按焦点、悬停，然后所有其他情况的优先级排序。

`NarratableEntry` 有三个方法：一个确定元素的优先级 (`#narrationPriority`)，一个确定是否朗读叙述 (`#isActive`)，最后一个提供叙述给关联的输出（语音或文字）(`#updateNarration`)。

!!! note Minecraft 中的所有小部件都是 `NarratableEntry`，因此在使用可用的子类型时通常不需要手动实现。

屏幕子类型

有了以上所有知识，就可以构建一个基本的屏幕了。为了更容易理解，屏幕的组件将按通常遇到的顺序说明。

首先，所有屏幕都接受一个表示屏幕标题的 `Component`。此组件通常由其子类型之一绘制到屏幕上。在基本屏幕中，它仅用于叙述消息。

```
// 在某些 Screen 子类中
public MyScreen(Component title) {
    super(title);
}
```

初始化

屏幕初始化后，会调用 `#init` 方法。`#init` 方法设置屏幕内的初始设置，从 `ItemRenderer` 和 `Minecraft` 实例到经过游戏缩放的相对宽度和高度。添加小部件或预计算相对坐标等任何设置都应在此方法中完成。如果游戏窗口调整大小，将通过调用 `#init` 方法重新初始化屏幕。

有三种方式向屏幕添加小部件，每种服务于不同的目的：

| 方法 | 描述 |
|-----------------------------------|----------------------|
| <code>#addWidget</code> | 添加可交互和可叙述但不可渲染的小部件。 |
| <code>#addRenderableOnly</code> | 添加仅渲染的小部件；不可交互也不可叙述。 |
| <code>#addRenderableWidget</code> | 添加可交互、可叙述且可渲染的小部件。 |

通常，`#addRenderableWidget` 会是最常用的。

```
// 在某些 Screen 子类中
@Override
protected void init() {
    super.init();
    // 添加小部件和预计算的值
    this.addRenderableWidget(new EditText(/* ... */));
}
```

屏幕 Tick

屏幕也使用 `#tick` 方法来执行一定程度的客户端逻辑以用于渲染。最常见的例子是 `EditText` 的光标闪烁。

```
// 在某些 Screen 子类中
@Override
public void tick() {
    super.tick();
    this.editBox.tick();
}
```

输入处理

由于屏幕是 `GuiEventListener` 的子类型，也可以重写输入处理器，例如处理特定按键的逻辑。

渲染屏幕

最后，屏幕通过作为 `Renderable` 子类型提供的 `#render` 方法进行渲染。如前所述，`#render` 方法每帧绘制屏幕需要渲染的所有内容，例如背景、小部件、提示框等。默认情况下，`#render` 方法仅将小部件渲染到屏幕。

屏幕内通常不由子类型处理的两种最常见渲染是背景和提示框。

背景可以使用 `#renderBackground` 渲染，其中一个方法在屏幕渲染时（当其后的世界无法显示时）接受一个 `v` 偏移用于选项背景。

提示框通过 `GuiGraphics#renderTooltip` 或 `GuiGraphics#renderComponentTooltip` 渲染，可以接受要渲染的文本组件、可选的自定义提示框组件以及提示框在屏幕上应渲染的 `x/y` 相对坐标。

```
// 在某些 Screen 子类中

// mouseX 和 mouseY 表示光标在屏幕上的缩放坐标
@Override
public void render(GuiGraphics graphics, int mouseX, int mouseY, float partialTick) {
    // 通常首先渲染背景
    this.renderBackground(graphics);
    // 在此处渲染小部件之前的内容（背景纹理）
    // 然后渲染小部件（如果这是 Screen 的直接子类）
    super.render(graphics, mouseX, mouseY, partialTick);
    // 在小部件之后渲染内容（提示框）
}
```

关闭屏幕

关闭屏幕时，有两个方法处理清理工作：`#onClose` 和 `#removed`。

`#onClose` 在用户输入关闭当前屏幕时调用。此方法通常用作回调，用于销毁和保存屏幕本身的任何内部进程。这包括向服务器发送数据包。

`#removed` 在屏幕即将更改并释放到垃圾回收器时调用。它处理任何尚未重置为屏幕打开前初始状态的内容。

```
// 在某些 Screen 子类中

@Override
public void onClose() {
    // 在此处停止任何处理器
    super.onClose();
}

@Override
public void removed() {
    // 在此处重置初始状态
    super.removed();
}
```

AbstractContainerScreen

如果屏幕直接附加到菜单，则应继承 `AbstractContainerScreen`。`AbstractContainerScreen` 充当菜单的渲染器和输入处理器，并包含用于与槽位同步和交互的逻辑。因此，通常只需要重写或实现两个方法即可拥有一个可工作的容器屏幕。同样，为了更容易理解，容器屏幕的组件将按通常遇到的顺序说明。

`AbstractContainerScreen` 通常需要三个参数：正在打开的容器菜单（由泛型 `T` 表示）、玩家物品栏（仅用于显示名称）和屏幕本身的标题。在此处，可以设置一些定位字段：

| 字段 | 描述 |
|------------------------------|--|
| <code>imageWidth</code> | 用于背景的纹理宽度。通常位于 256 x 256 的 PNG 内部，默认为 176。 |
| <code>imageHeight</code> | 用于背景的纹理高度。通常位于 256 x 256 的 PNG 内部，默认为 166。 |
| <code>titleLabelX</code> | 屏幕标题渲染的相对 x 坐标。 |
| <code>titleLabelY</code> | 屏幕标题渲染的相对 y 坐标。 |
| <code>inventoryLabelX</code> | 玩家物品栏名称渲染的相对 x 坐标。 |
| <code>inventoryLabelY</code> | 玩家物品栏名称渲染的相对 y 坐标。 |

!!! important 在前面的部分中提到，预计算的相对坐标应在 `#init` 方法中设置。这仍然成立，因为此处提到的值不是预计算的坐标，而是静态值和相对化坐标。

`image` 值是静态且不变的，因为它们代表背景纹理大小。为了简化渲染，`#init` 方法预计算了两个额外值（``leftPos`` 和 ``topPos``）。

``leftPos`` 和 ``topPos`` 也可以作为渲染背景的便利方式，因为它们已经代表了传递给 `#blit` 方法的位置。

```
// 在某些 AbstractContainerScreen 子类中
public MyContainerScreen(MyMenu menu, Inventory playerInventory, Component title) {
    super(menu, playerInventory, title);

    this.titleLabelX = 10;
    this.inventoryLabelX = 10;
}
```

菜单访问

由于菜单被传入屏幕，菜单内已同步的任何值（通过槽位、数据槽位或自定义系统）现在都可以通过 `menu` 字段访问。

容器 Tick

当玩家存活并正在查看屏幕时，容器屏幕在 `#tick` 方法中通过 `#containerTick` 进行 tick。这实际上取代了容器屏幕中的 `#tick`，其最常见的用途是对配方书进行 tick。

```
// 在某些 AbstractContainerScreen 子类中
@Override
protected void containerTick() {
    super.containerTick();
    // 在此处进行 tick
}
```

渲染容器屏幕

容器屏幕通过三个方法进行渲染：`#renderBg`（渲染背景纹理）、`#renderLabels`（在背景之上渲染文本）和 `#render`（包含了前两个方法，并额外提供了灰色背景和提示框）。

从 `#render` 开始，最常见的重写（通常也是唯一的情况）添加背景、调用 `super` 渲染容器屏幕，最后在其上渲染提示框。

```
// 在某些 AbstractContainerScreen 子类中
@Override
public void render(GuiGraphics graphics, int mouseX, int mouseY, float partialTick) {
    this.renderBackground(graphics);
    super.render(graphics, mouseX, mouseY, partialTick);
    this.renderTooltip(graphics, mouseX, mouseY);
}
```

在 `super` 内部，调用 `#renderBg` 来渲染屏幕背景。最标准的表示使用三个方法调用：两个用于设置，一个用于绘制背景纹理。

```
// 在某些 AbstractContainerScreen 子类中

private static final ResourceLocation BACKGROUND_LOCATION = new ResourceLocation(MOD_ID, "textures/gui/container/my_container_screen.png");

@Override
```

```
protected void renderBg(GuiGraphics graphics, float partialTick, int mouseX, int mouseY) {
    graphics.blit(BACKGROUND_LOCATION, this.leftPos, this.topPos, 0, 0, this.imageWidth, this.imageHeight);
}
```

最后，调用 `#renderLabels` 在背景之上、提示框之下渲染文本。

```
// 在某些 AbstractContainerScreen 子类中
@Override
protected void renderLabels(GuiGraphics graphics, int mouseX, int mouseY) {
    super.renderLabels(graphics, mouseX, mouseY);
    graphics.drawString(this.font, this.label, this.labelX, this.labelY, 0x404040);
}
```

!!! note 渲染标签时，你**不需要**指定 `leftPos` 和 `topPos` 偏移。这些已经在 `PoseStack` 中进行了平移，因此此方法内的所有内容都是相对于这些坐标绘制的。

注册 AbstractContainerScreen

要将 `AbstractContainerScreen` 与菜单一起使用，需要注册它。这可以通过在 `FMLClientSetupEvent`（在**模组事件总线**上）中调用 `MenuScreens#register` 来完成。

```
// 事件在模组事件总线上监听
private void clientSetup(FMLClientSetupEvent event) {
    event.enqueueWork(
        // 假设 RegistryObject<MenuType<MyMenu>> MY_MENU
        // 假设 MyContainerScreen<MyMenu> 接受三个参数
        () -> MenuScreens.register(MY_MENU.get(), MyContainerScreen::new)
    );
}
```

!!! warning `MenuScreens#register` 不是线程安全的，因此需要在并行调度事件提供的 `#enqueueWork` 内部调用。

根变换

在模型 JSON 顶层添加 `transform` 条目，告知加载器应在方块模型的 `blockstate` 文件中的旋转之前，或在物品模型的显示变换之前，对所有几何体应用变换。该变换可通过 `IUnbakedGeometry#bake()` 中的 `IGeometryBakingContext#getRootTransform()` 获取。

自定义模型加载器可以完全忽略此字段。

根变换可以以两种格式指定：

- 矩阵格式**：包含 `matrix` 条目，含原始变换矩阵 (3×4 , 行主序): `js "transform": { "matrix": [ [0,0,0,0], [0,0,0,0], [0,0,0,0] ] }`
- 元素格式**：包含以下条目的任意组合：
 - `origin`：旋转/缩放原点。值: "corner"(0,0,0) / "center"(.5,.5,.5) / "opposing-corner"(1,1,1) 或 `[x,y,z]`
 - `translation`：相对平移 `[x,y,z]`
 - `rotation/left_rotation`：缩放前的旋转（绕平移后原点）
 - `scale`：缩放 `[x,y,z]`
 - `right_rotation/post_rotation`：缩放后的旋转

应用顺序: `translation` → `left_rotation` → `scale` → `right_rotation`, 最后移动到原点。

```
{ "transform": { "origin": "center", "translation": [0, 0.5, 0], "rotation": { "y": 45 } } }
```

旋转格式支持：单轴 `{ "x": 90 }`、轴数组 `[ { "x": 90 }, { "y": 45 } ]`、欧拉角 `[ 90, 180, 45 ]`、四元数 `[ 0.382, 0, 0, 0.924 ]`。

渲染类型

在 JSON 顶层添加 `render_type` 条目，告知加载器模型应使用的渲染类型。如果未指定，加载器会选择使用的渲染类型，通常回退到 `ItemBlockRenderTypes#getRenderLayers()` 返回的渲染类型。

自定义模型加载器可以完全忽略此字段。

!!! note 自 1.19 起，对于方块，这优先于通过 `ItemBlockRenderTypes#setRenderLayer()` 设置适用渲染类型的已弃用方法。

使用玻璃纹理的 cutout 方块模型示例:

```
{
  "render_type": "minecraft:cutout",
  "parent": "block/cube_all",
  "textures": {
    "all": "block/glass"
  }
}
```

原版值

以下是 Forge 提供的具有相应区块和实体渲染类型的选项:

- minecraft:solid —完全实心 (石头)
- minecraft:cutout —完全透明或不透明 (玻璃)
- minecraft:cutout_mipped —带 mipmapping 的 cutout (树叶)
- minecraft:cutout_mipped_all —物品也应用 mipmapping 的 cutout
- minecraft:translucent —半透明 (染色玻璃)
- minecraft:tripwire —需要天气渲染目标的特殊类型 (绊线钩)

自定义值

自定义命名渲染类型可以在 RegisterNamedRenderTypesEvent (模组事件总线) 中注册。由区块渲染类型 + 实体渲染类型 (必需) + 实体渲染类型 *Fabulous!* 模式 (可选) 组成。

```
event.register("special_cutout", RenderType.cutout(), Sheets.cutoutBlockSheet());
event.register("special_translucent", RenderType.translucent(), Sheets.translucentCullBlockSheet(),
Sheets.translucentItemSheet());
```

在 JSON 中引用为 <your_mod_id>:special_cutout。

部件可见性

在模型 JSON 顶层添加 visibility 条目, 控制模型不同部分的可见性, 决定它们是否应被烘焙到最终的 BakedModel 中。“部件”定义取决于模型加载器。Forge 的复合模型加载器和 OBJ 模型加载器 使用此功能。

可见性条目指定为 "part name": boolean:

```
// 复合模型: part_two 默认不可见
{ "loader": "forge:composite", "children": { "part_one": {...}, "part_two": {...} }, "visibility": { "part_two": false } }

// 子模型 1: 仅 part_two 可见
{ "parent": "mymod:mycompositemodel", "visibility": { "part_one": false, "part_two": true } }

// 子模型 2: part_two 可见
{ "parent": "mymod:mycompositemodel", "visibility": { "part_two": true } }
```

可见性判定: 先查自身, 没有再递归查父级, 直到找到条目或没有父级可查 (默认为 true)。

面数据

在原版“elements”模型中, 关于元素面的额外数据可以在元素级别或面级别指定。未指定自己面数据的面将回退到元素的面数据, 如果元素级别也未指定, 则使用默认值。

要使用此扩展于生成的物品模型, 模型必须通过 forge:item_layers 模型加载器加载, 因为原版物品模型生成器未被扩展以读取这些额外数据。

所有面数据的值都是可选的。

Elements 模型

在原版“elements”模型中, 面数据适用于指定了该数据的面, 或未指定面数据的元素的所有面。

!!!note 如果在面上指定了 `forge_data`，它将不会从元素级别的 `forge_data` 声明继承任何参数。

额外数据可以通过以下示例中的两种方式指定：

```
{
  "elements": [
    {
      "forge_data": {
        "color": "0xFFFF0000",
        "block_light": 15,
        "sky_light": 15,
        "ambient_occlusion": false
      },
      "faces": {
        "north": {
          "forge_data": {
            "color": "0xFFFF0000",
            "block_light": 15,
            "sky_light": 15,
            "ambient_occlusion": false
          }
        }
      }
    }
  ]
}
```

生成的物品模型

在使用 `forge:item_layers` 加载器生成的物品模型中，面数据为每个纹理层指定，并应用于所有几何体（正面/背面四边形和边缘四边形）。

`forge_data` 字段必须位于模型 JSON 的顶层，每个键值对将一个面数据对象关联到一个层索引。

在以下示例中，第 1 层将被着色为红色并以最大亮度发光：

```
{
  "textures": {
    "layer0": "minecraft:item/stick",
    "layer1": "minecraft:item/glowstone_dust"
  },
  "forge_data": {
    "1": {
      "color": "0xFFFF0000",
      "block_light": 15,
      "sky_light": 15,
      "ambient_occlusion": false
    }
  }
}
```

参数

颜色

使用 `color` 条目指定颜色值将把该颜色作为着色应用于四边形。默认值为 `0xFFFFFFFF`（白色，完全不透明）。颜色必须采用 ARGB 格式，打包为 32 位整数，可以指定为十六进制字符串（"`0xAARRGGBB`"）或十进制整数（JSON 不支持十六进制整数）。

!!! warning 四个颜色分量会与纹理的像素相乘。省略 `alpha` 分量相当于将其设为 0，这将使几何体完全透明。

如果颜色值是恒定的，这可以用作 `BlockColor` 和 `ItemColor` 着色的替代方案。

方块光和天空光

通过 `block_light` 和 `sky_light` 条目分别指定方块光和/或天空光值，将覆盖四边形的相应光照值。两个值默认为 0。值必须在 0-15（含）范围内，当面被渲染时，它们被视为对应光照类型的最小值，意味着世界中该光照类型的更高值将覆盖指定的值。

指定的光照值纯粹是客户端的，既不影响服务器的光照级别，也不影响周围方块的亮度。

环境光遮蔽

指定 `ambient_occlusion` 标志将为四边形配置 AO。默认值为 `true`。此标志的行为等同于原版格式的顶层 `ambientocclusion` 标志。

!!! note 如果顶层 AO 标志设置为 `false`，在元素或面上将此标志指定为 `true` 将无法覆盖顶层标志。

自定义模型加载器

“模型”是一个形状，由 `IGeometryLoader` 在运行时加载为 `IUnbakedGeometry`，最终烘焙为 `BakedModel`。

Forge 提供的内置加载器：

- **WaveFront OBJ** (`forge:obj`) —加载 `.obj` 文件，支持 `flip_v` 翻转 V 轴
- **桶模型** —原版桶的专用加载器
- **复合模型** (`forge:composite`) —组合多个子模型
- **不同渲染层** —使用 `render_type` 指定
- **builtin/generated 重实现** —原版物品模型生成器的 Forge 版本

!!! warning 指定 `loader` 后，`elements` 条目将被忽略（除非被自定义加载器消费）。

WaveFront OBJ 模型示例：

```
{
  "loader": "forge:obj",
  "flip_v": true,
  "model": "examplemod:models/block/model.obj",
  "textures": { "texture0": "minecraft:block/dirt", "particle": "minecraft:block/dirt" }
}
```

BakedModel

`BakedModel` 是调用 `UnbakedModel#bake` (原版) 或 `IUnbakedGeometry#bake` (自定义) 的结果。代表经过优化、准备发送到 GPU 的几何体。

核心方法：

- **getQuads** —主要方法。返回 `BakedQuad` 列表（包含底层顶点数据）。作为方块渲染时 `BlockState` 非 `null`；作为物品渲染时 `BlockState` 为 `null`。Direction 用于面剔除。此方法调用极频繁，应大量使用缓存。
- **getOverrides** —返回 `ItemOverrides`，仅物品渲染时使用。
- **useAmbientOcclusion** —是否使用环境光遮蔽。
- **isGui3d** —物品渲染时是否显示为“立体”。
- **isCustomRenderer** —返回 `true` 时回退到 `BlockEntityWithoutLevelRenderer` 渲染。
- **getParticleIcon** —粒子纹理。
- **applyTransform** —处理物品显示变换（替代已弃用的 `getTransforms`）。

!!! note `BakedQuad` 顶点原点为西北角底部，建议按逆时针顺序提供。

变换

当 `BakedModel` 作为物品渲染时，可以根据所处的变换上下文应用特殊处理。“变换”指模型被渲染时的上下文，由 `ItemDisplayContext` 枚举表示。

`ItemDisplayContext` 值：- `NONE` —无上下文（默认），Forge 用于 `ENTITYBLOCK_ANIMATED` 方块 - `THIRD_PERSON_*` / `FIRST_PERSON_*` —第三人称/第一人称手持 - `HEAD` —头盔槽佩戴 - `GUI` —屏幕中渲染 - `GROUND` —掉落物实体 - `FIXED` —物品展示框

原版方式（已弃用）：`BakedModel#getTransforms` → `ItemTransforms` → `ItemTransform`（含旋转/平移/缩放）。

Forge 方式（推荐）：实现 `#applyTransform(ItemDisplayContext, PoseStack, boolean leftHand)` → 返回新的 `BakedModel`。更灵活，可以返回完全不同的模型（如纸在手中是平的，在地上是皱的）。

默认返回 `ItemTransforms#NO_TRANSFORMS` 并实现 `#applyTransform` 即可。

ItemOverrides

ItemOverrides 为 BakedModel 提供根据 ItemStack 状态切换模型的能力，是实现动态模型的关键。函数签名：(BakedModel, ItemStack, ClientLevel, LivingEntity, int) → BakedModel。

核心类

- **ItemOverrides** —持有 BakedOverride 列表，通过 #resolve 方法解析物品状态并返回对应模型。
- **BakedOverride** —表示一个原版物品覆盖，包含属性匹配器 (predicate) 和满足条件时使用的目标模型。

原版 JSON 格式

```
{
  "overrides": [
    { "predicate": { "example:prop": 0.5 }, "model": "example:item/model1" },
    { "predicate": { "example:prop": 1 }, "model": "example:item/model2" }
  ]
}
```

资源包

资源包 通过 assets 目录自定义客户端资源，包括纹理、模型、声音、本地化等。你的模组（以及 Forge 本身）也可以拥有资源包。

资源包存储在项目的 resources 目录中。assets 目录包含包的内容，包本身由 pack.mcmeta 定义。模组可以有多个资源域。

详见 Minecraft Wiki 资源包教程。

模型

模型系统是 Minecraft 赋予方块和物品形状的方式。模型通过 ResourceLocation 与纹理关联，存储在 ModelManager 中。

方块使用 ModelResourceLocation (注册名 + 序列化的 BlockState)，物品使用注册名 + inventory。

纹理

- UV 坐标 (0,0) 为**左上角**，范围 0-16
- 纹理应为正方形，边长为 2 的幂 (1x1, 2x2, 16x16, 128x128 等)
- 非正方形纹理仅用于动画

注意

JSON 模型仅支持长方体元素，无法表达三角形等复杂形状。如需复杂模型，需使用其他格式（如 OBJ）。

纹理着色

许多方块和物品的纹理颜色会根据位置或属性变化（如草）。模型支持在面上指定“着色索引” (tint index)，由 BlockColor 和 ItemColor 处理。

BlockColor / ItemColor

均为单方法接口，接受 tintIndex 参数，返回颜色乘数 (int，按 ARGB 格式打包)。每个像素的计算：(int)(base * multiplier / 255.0)

注册颜色处理器

- RegisterColorHandlersEvent\$Block (模组事件总线)：event.register(myBlockColor, block1, block2...)
- RegisterColorHandlersEvent\$Item (模组事件总线)：event.register(myItemColor, item1, item2...)

注意：注册 BlockColor 不会自动为 BlockItem 着色，需要单独注册 ItemColor。

内置模型

物品使用 builtin/generated 模型时，每层 (layer0, layer1...) 的着色索引对应其层索引。

物品属性

物品属性将物品的“属性”暴露给模型系统。例如弓的拉弓程度，用于选择模型创建动画。

注册物品属性

`ItemProperties#register(item, name, propertyFunction)` —在 `FMLClientSetupEvent` 中调用，需使用 `#enqueueWork`。- `item`: 目标物品 - `name`: 属性名称，建议使用模组 ID 作为命名空间 - `propertyFunction`: `(ItemStack, ClientLevel, LivingEntity, int) → float`

`ItemProperties#registerGeneric` —为所有物品注册属性。

使用覆盖

在模型 JSON 中使用 `overrides` 数组，谓词匹配所有**大于等于**给定值的属性。

```
{
  "overrides": [
    { "predicate": { "examplemod:power": 0.75 }, "model": "examplemod:item/example_powered" }
  ]
}
```

多个匹配时选择列表中的最后一个。

数据包

自 1.13 起，Mojang 在原版游戏中添加了数据包。数据包通过 `data` 目录修改逻辑服务器的文件，包括进度、战利品表、结构、配方、标签等。

数据包存储在项目的 `resources` 目录中的 `data` 文件夹内。模组可以有多个数据域。

详见 Minecraft Wiki 数据包教程。

配方

配方是将一些对象转换为其他对象的机制。原版系统主要用于物品转换，但可以扩展为使用任何对象。

数据驱动

大多数原版配方由 JSON 数据驱动。通过 `RecipeManager` 加载和存储。- `getRecipeFor` —获取第一个匹配的配方 - `getRecipesFor` —获取所有匹配的配方 - Forge 提供 `RecipeWrapper` (包装 `IItemHandler → Container`)

Forge 扩展

- **物品堆结果** —支持完整的 `ItemStack` 作为结果 (含 `count` 和 `nbt`)
- **条件配方** —见 `conditional` 页面
- **更大合成网格** —`ShapedRecipe#setCraftingSize` (注意线程安全)
- **额外原料类型** —见 `ingredients` 页面

自定义配方

每个配方定义由三个组件组成：

1. **Recipe** —持有数据和执行逻辑。实现 `#matches`、`#assemble`、`#getType`、`#getSerializer`
2. **RecipeType** —定义配方所属类别 (如 `SMELTING`、`CRAFTING`)，需要时注册新类型
3. **RecipeSerializer** —编解码配方 JSON 和网络通信。实现 `fromJson`、`toNetwork`、`fromNetwork`

JSON 格式

```
{
  "type": "examplemod:example_serializer",
  "input": { /* ingredient */ },
  "data": 0,
  "output": { /* stack */ }
}
```

非物品逻辑

如果配方不涉及物品变换，可在自定义 Recipe 中添加额外方法，使用 `RecipeManager#getAllRecipesFor` 获取后手动检查。

数据生成

自定义配方可以通过创建 `FinishedRecipe` 实现数据生成。

原料 (Ingredient)

`Ingredient` 是物品输入的谓词处理器，检查 `ItemStack` 是否满足配方输入条件。

Forge 提供的类型

- `forge:nbt` (`StrictNBTIngredient`) —精确匹配物品、伤害和标签
- `forge:partial_nbt` (`PartialNBTIngredient`) —仅匹配指定的标签键
- `forge:intersection` —必须匹配所有子原料 (AND)
- `forge:difference` —必须在 `base` 中但不在 `subtracted` 中 (SUB)
- `CompoundIngredient` —无 `type` 字段，原料数组 OR 组合

自定义原料

1. 继承 `AbstractIngredient`，实现 `test`、`isSimple`、`getSerializer`、`toJson`
2. 实现 `IIngredientSerializer` (`parse JSON/网络`、`write`)
3. 使用 `CraftingHelper#register` 在 `RegisterEvent` 或 `FMLCommonSetupEvent` 中注册

非数据包配方

部分配方子系统尚未迁移到数据驱动方式，仍需要在代码中实现。

酿造配方

通过 `BrewingRecipeRegistry#addRecipe` 在 `FMLCommonSetupEvent` 中添加（注意线程安全）。- 默认实现：输入 + 催化剂 + 输出
- 也可提供 `IBrewingRecipe` 实例（实现 `isInput`、`isIngredient`、`getOutput`）

铁砧配方

使用 `AnvilUpdateEvent`（Forge 事件总线），包含左右物品、输出、经验等级和材料数量。可通过取消事件阻止输出。

织布机配方

织布机负责将染料和图案应用于旗帜。自定义旗帜图案通过注册 `BannerPattern` 实现。- `minecraft:no_item_required` 标签中的图案可直接在织布机中使用 - 不在该标签中的图案需要配套的 `BannerPatternItem`

战利品表

战利品表是决定各种动作或场景发生时应该产生什么结果的逻辑文件。原版系统主要用于物品生成，但可以扩展为执行任意操作。

数据驱动

大多数原版战利品表由 JSON 数据驱动。详见 Minecraft Wiki。

使用战利品表

- ResourceLocation 指向 data/<namespace>/loot_tables/<path>.json
- 通过 MinecraftServer#getLootData → LootDataResolver#getLootTable 获取
- 使用 LootParams\$Builder 构建参数, LootContext\$Builder 构建上下文
- 方法: getRandomItemsRaw、getRandomItems、fill

Forge 扩展

- **LootTableLoadEvent** —加载时触发, 取消则加载空表 (不应用来修改掉落, 应使用 GLM)
- **战利品池命名** —使用 name 键命名
- **战利品等级** —LootingLevelEvent 影响战利品等级
- **烧炼多物品** —SmeltItemFunction 现在返回实际数量的物品
- **战利品表 ID 条件** —forge:loot_table_id 条件
- **工具操作条件** —forge:can_tool_perform_action 条件

全局战利品修改器

全局战利品修改器 (GLM) 是一种数据驱动的方法, 用于处理掉落物修改, 无需覆盖数十到数百个原版战利品表。

需要 4 个步骤

1. **data/forge/loot_modifiers/global_loot_modifiers.json** —声明所有加载的修改器列表 (entries + replace)
2. **序列化 JSON** —data/<namespace>/loot_modifiers/<name>.json, 包含 type (Codec 注册名)、conditions (激活条件) 和自定义属性
3. **IGlobalLootModifier 实现** —建议继承 LootModifier, 实现 #doApply 和 #codec
4. **Codec** —注册 Codec<? extends IGlobalLootModifier>, LootModifier#codecStart 简化条件部分

JSON 示例

```
{
  "type": "examplemod:example_loot_modifier",
  "conditions": [ /* Loot table conditions */ ],
  "prop1": "val1", "prop2": 10, "prop3": "minecraft:dirt"
}
```

注意

- global_loot_modifiers.json 必须在 forge 命名空间下
- GLM 是叠加的 (类似标签), 而非后加载覆盖
- 多个 GLM 按注册顺序依次处理掉落物

标签

标签是游戏中对象的广义集合, 用于将相关内容分组并提供快速成员检查。

声明标签

标签在数据包中声明。TagKey<Block> 指向 /data/<modid>/tags/blocks/<path>.json。- replace: true —替换而非追加 - required: false 一条目不存在时不报错 - Forge 扩展: remove 数组—精细移除

代码中使用

- 通过 `*Tags#create`、`ITagManager#createTagKey`、`DeferredRegister#createTagKey`、`TagKey#create` 创建标签包装器
- 对象通过 `Holder#is(tag)` 或 `ITag#contains` 检查是否属于标签

约定

- 存在原版/Forge 标签时优先使用
- 公共分组使用 `forge` 命名空间
- 物品/方块分组使用复数名称 (`minecraft:logs`)
- 物品标签按类型分目录 (`forge:ingots/iron`)

OreDictionary 迁移

- 配方中可直接使用标签
- 代码中使用 `stack.is(tag)` 替代旧版 OreDictionary 检查

进度

进度是玩家可以达成的任务，由 JSON 数据驱动。

进度条件

- 使用 `criteria` 对象定义条件
- `requirements` 定义条件组合方式 (AND/OR)
- 条件触发器定义在 `CriteriaTriggers` 中

自定义条件触发器

1. 继承 `AbstractCriterionTriggerInstance` — 持有条件数据，实现 `#matches` 和 `#serializeToJson`
2. 继承 `SimpleCriterionTrigger<T>` — 指定 `#getId`，实现 `#createInstance`，提供 `#trigger` 方法
3. 在 `FMLCommonSetupEvent` 中使用 `CriteriaTriggers#register` 注册 (注意线程安全)

进度奖励

经验值、战利品表、配方 (配方书)、函数。

```
"rewards": { "experience": 10, "loot": [...], "recipes": [...], "function": "..." }
```

条件加载数据

允许在模组间实现软依赖的条件加载系统。目前支持配方和进度。

JSON 格式

使用 `"type": "forge:conditional"`，内嵌 `recipes / advancements` 数组，每个条目包含 `conditions` (AND 组合) 和对应的 `recipe/advancement`。

内置条件

- `forge:true / forge:false` — 布尔常量
- `forge:not / forge:and / forge:or` — 布尔运算
- `forge:mod_loaded` — 检查模组是否加载
- `forge:item_exists` — 检查物品是否注册
- `forge:tag_empty` — 检查标签是否为空

自定义条件

实现 `ICondition + IConditionSerializer`, 在 `RegisterEvent` 或 `FMLCommonSetupEvent` 中使用 `CraftingHelper#register` 注册。

数据生成器

数据生成器是一种以编程方式生成模组资源和数据的方式。它允许在代码中定义这些文件的内容并自动生成，无需关心具体细节。

数据生成器系统由主类 `net.minecraft.data.Main` 加载。可以通过不同的命令行参数来定制收集哪些模组的数据、考虑哪些现有文件等。负责数据生成的类是 `net.minecraft.data.DataGenerator`。

MDK 的 `build.gradle` 中默认添加了 `runData` 任务用于运行数据生成器。

现有文件

所有对未通过数据生成生成的纹理或其他数据文件的引用，都必须引用系统上现有的文件。这是为了确保所有引用的纹理都在正确的位置，以便发现和纠正拼写错误。

`ExistingFileHelper` 是负责验证这些数据文件是否存在的类。可以从 `GatherDataEvent#getExistingFileHelper` 获取其实例。

`--existing <folderpath>` 参数允许在验证文件是否存在时使用指定的文件夹及其子文件夹。此外，`--existing-mod <modid>` 参数允许使用已加载模组的资源进行验证。默认情况下，只有原版数据包和资源可用于 `ExistingFileHelper`。

生成器模式

数据生成器可以配置运行 4 种不同的数据生成，通过命令行参数配置，并可通过 `GatherDataEvent#include***` 方法检查。

- **客户端资源 (Client Assets)**
 - 在 `assets` 中生成仅客户端的文件：方块/物品模型、blockstate JSON、语言文件等。
 - `--client, #includeClient`
- **服务端数据 (Server Data)**
 - 在 `data` 中生成仅服务端的文件：配方、进度、标签等。
 - `--server, #includeServer`
- **开发工具 (Development Tools)**
 - 运行一些开发工具：SNBT 与 NBT 相互转换等。
 - `--dev, #includeDev`
- **报告 (Reports)**
 - 导出所有已注册的方块、物品、命令等。
 - `--reports, #includeReports`

`--all` 可以包含所有生成器。

数据提供者 (Data Providers)

数据提供者是实际定义生成哪些数据的类。所有数据提供者实现 `DataProvider`。Minecraft 为大多数资源和数据提供了抽象实现，模组开发者只需扩展并重写指定的方法。

`GatherDataEvent` 在创建数据生成器时在模组事件总线上触发，可以从事件中获取 `DataGenerator`。使用 `DataGenerator#addProvider` 创建并注册数据提供者。

客户端资源

- `LanguageProvider` —用于语言字符串；实现 `#addTranslations`
- `SoundDefinitionsProvider` —用于 `sounds.json`；实现 `#registerSounds`
- `ModelProvider<?>` —用于模型；实现 `#registerModels`
 - `ItemModelProvider` —用于物品模型
 - `BlockModelProvider` —用于方块模型
- `BlockstateProvider` —用于 blockstate JSON 及其方块和物品模型；实现 `#registerStatesAndModels`

服务端数据

- GlobalLootModifierProvider —用于全局战利品修改器；实现 #start
- DatapackBuiltinEntriesProvider —用于数据包注册表对象；向构造函数传入 RegistrySetBuilder
- LootTableProvider —用于战利品表；向构造函数传入 LootTableProvider\$SubProviderEntry
- RecipeProvider —用于配方及其解锁进度；实现 #buildRecipes
- TagsProvider —用于标签；实现 #addTags
- AdvancementProvider —用于进度；向构造函数传入 AdvancementSubProvider

模型生成

模型和方块状态可以通过继承相关提供者来生成。

模型文件 (ModelFile)

- ExistingModelFile —通过 ExistingFileHelper 检查模型是否已存在
- UncheckedModelFile —假设模型存在（不推荐使用）

模型构建器 (ModelBuilder)

ModelBuilder 表示一个待生成的 ModelFile。设置父模型、面、纹理、变换等。

- BlockModelBuilder —方块模型。额外支持 rootTransform（根变换）
- ItemModelBuilder —物品模型。额外支持 overrides（覆盖）

模型提供者

- BlockModelProvider —在 models/block 中生成方块模型
- ItemModelProvider —在 models/item 中生成物品模型。#basicItem 使用 item/generated 模板

方块状态提供者 (BlockStateProvider)

负责生成 blockstate JSON、方块模型和物品模型。实现 #registerStatesAndModels。

- 变体 (Variant): VariantBlockStateBuilder —每个 BlockState 只能匹配一个变体
- 多部分 (Multipart): MultiPartBlockStateBuilder —多个部分可以同时叠加（如红石线）

自定义加载器构建器

Forge 提供的内置加载器数据生成：- DynamicFluidContainerModelBuilder —桶模型 - CompositeModelBuilder —复合模型 - ItemLayersModelBuilder —多层物品模型 - SeparateTransformsModelBuilder —按变换切换模型 - ObjModelBuilder —OBJ 模型

```
builder.customLoader(ObjModelBuilder::begin)
    .modelLocation(modLoc("models/block/model.obj"))
    .flipV(true)
    .end()
    .texture("particle", mcLoc("block/dirt"));
```

语言文件生成

通过继承 LanguageProvider 并实现 #addTranslations 来生成模组的语言文件。每个 LanguageProvider 子类代表一个独立的语言环境（en_us 代表美式英语，es_es 代表西班牙语等）。实现后，必须添加到 DataGenerator。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeClient(),
        output -> new MyLanguageProvider(output, MOD_ID, "en_us")
    );
}
```

LanguageProvider 是一个字符串映射，将翻译键映射到本地化名称。使用 #add 添加翻译键映射，也有针对 Block、Item、ItemStack、Enchantment、MobEffect、EntityType 的便捷方法。

```
// 在 LanguageProvider#addTranslations 中
this.addBlock(EXAMPLE_BLOCK, " 示例方块");
this.add("object.examplemod.example_object", " 示例对象");
```

!!! tip 包含非美式英语字母数字字符的本地化名称可以直接提供。提供者会自动将字符转换为对应的 unicode 编码供游戏读取。

声音定义生成

通过继承 SoundDefinitionsProvider 并实现 #registerSounds 生成 sounds.json。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeClient(),
        output -> new MySoundDefinitionsProvider(output, MOD_ID, event.getExistingFileHelper())
    );
}
```

方法

- #add(soundName, definition) —添加声音定义
- #definition() —创建 SoundDefinition
 - .with(sound...) —添加声音文件
 - .subtitle(key) —设置翻译键
 - .replace(true) —替换而非追加
- #sound(location, type) —创建 SoundDefinition\$Sound
 - type: SOUND (声音文件) 或 EVENT (其他声音事件)
 - .volume(v) / .pitch(p) / .weight(w) / .stream() / .attenuationDistance(d) / .preload()

```
this.add(EXAMPLE_SOUND_EVENT, definition()
    .subtitle("sound.examplemod.example_sound")
    .with(
        sound(new ResourceLocation(MODID, "example_sound_1")).weight(4).volume(0.5),
        sound(new ResourceLocation(MODID, "example_sound_2")).stream()
    )
);
```

配方生成

通过继承 RecipeProvider 并实现 #buildRecipes 来生成配方。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(event.includeServer(), MyRecipeProvider::new);
}
```

配方构建器

- ShapedRecipeBuilder —有序合成: .shaped(category, result) → .pattern(str) → .define(char, item) → .unlockedBy(name, trigger) → .save(writer)
- ShapelessRecipeBuilder —无序合成: .shapeless(category, result) → .requires(item) → .unlockedBy(...) → .save(writer)
- SimpleCookingRecipeBuilder —烧炼/高炉/烟熏/营火: .smelting(input, category, result, exp, time)
- SingleItemRecipeBuilder —切石机: .stonecutting(input, category, result)
- SmithingTransformRecipeBuilder —锻造台转换: .smithing(template, base, addition, category, result)
- SmithingTrimRecipeBuilder —锻造台盔甲纹饰: .smithingTrim(template, base, addition, category)
- SpecialRecipeBuilder —特殊动态配方: .special(serializer), 仅生成空 JSON

条件配方

通过 `ConditionalRecipe.builder() → .addCondition(cond) → .addRecipe(recipe) → .generateAdvancement() → .build(writer, name)` 实现。

`IConditionBuilder` 接口简化条件构建: `and()`, `or()`, `not()`, `itemExists(mod, path)`, `FALSE()` 等。

自定义序列化器

创建实现 `FinishedRecipe` 的构建器, 编码配方数据和解锁进度的 JSON。

战利品表生成

通过构建 `LootTableProvider` 并提供 `LootTableProvider$SubProviderEntry` 来生成战利品表。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeServer(),
        output -> new MyLootTableProvider(output, Collections.emptySet(), List.of(subProvider1, subProvider2))
    );
}
```

核心组件

- **LootTableSubProvider** —实现 `#generate(writer)`, 调用 `writer#accept` 添加战利品表
- **BlockLootSubProvider** —方块战利品表, 需实现 `#getKnownBlocks` 和 `#generate`
- **EntityLootSubProvider** —实体战利品表, 需实现 `#getKnownEntityTypes` 和 `#generate`

战利品表构建器

`LootTable#lootTable() → .withPool(pool) → .apply(function) LootPool#lootPool() → .add(entry) → .when(condition) → .apply(function) → .setRolls(n) → .setBonusRolls(n)`

- **条目 (LootPoolEntryContainer)**: 定义操作, 通常生成物品
- **条件 (LootItemCondition)**: 定义执行条件, 可 `#or` 组合, `#invert` 取反
- **函数 (LootItemFunction)**: 修改执行结果
- **数值提供者 (NumberProvider)**: 决定战利品池执行次数

标签生成

通过继承 `TagsProvider` 并实现 `#addTags` 来生成标签。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeServer(),
        output -> new MyBlockTagsProvider(output, event.getLookupProvider(), MOD_ID, event.getExistingFileHelper())
    );
}
```

TagAppender 方法

- `#add(object)` —通过资源键添加对象
- `#addOptional(name)` —通过名称添加 (可选依赖)
- `#addTag(tag)` —添加整个标签
- `#addOptionalTag(name)` —通过名称添加标签 (可选)
- `#replace(true)` —替换而非追加
- `#remove(object)` —从标签中移除

现有标签提供者

| 注册表对象 | 标签提供者 |
|------------|------------------------|
| Block | BlockTagsProvider* |
| Item | ItemTagsProvider |
| EntityType | EntityTypeTagsProvider |
| Fluid | FluidTagsProvider |
| Biome | BiomeTagsProvider |
| 更多 | ... |

* Forge 添加

ItemTagsProvider#copy(blockTag, itemTag) —— 键复制方块标签到物品标签。

自定义标签提供者

继承 TagsProvider, 传入注册表键。若希望使用对象本身添加标签 (而非 ResourceKey), 继承 IntrinsicHolderTagsProvider。

进度生成

通过构建 AdvancementProvider 并提供 AdvancementSubProvider 来生成进度。Forge 提供 ForgeAdvancementProvider 以获得更好的集成。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeServer(),
        output -> new ForgeAdvancementProvider(output, event.getLookupProvider(), event.getExistingFileHelper(),
            List.of(myGenerator))
    );
}
```

Advancement\$Builder 用法: - .parent(adv) — 设置父进度 - .display(...) — 设置显示信息 - .rewards(...) — 设置奖励 - .addCriterion(name, trigger) — 添加解锁条件 - .requirements(...) — 条件组合方式 (AND/OR) - .save(writer, name, efh) — 保存

```
Advancement example = Advancement.Builder.advancement()
    .addCriterion("example_criterion", triggerInstance)
    .save(writer, name, existingFileHelper);
```

全局战利品修改器生成

通过继承 GlobalLootModifierProvider 并实现 #start 来生成全局战利品修改器。使用 #add(name, instance) 添加 GLM。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeServer(),
        output -> new MyGlobalLootModifierProvider(output, MOD_ID)
    );
}

// 在 GlobalLootModifierProvider#start 中
this.add("example_modifier", new ExampleModifier(
    new LootItemCondition[] { WeatherCheck.weather().setRaining(true).build() },
    "val1", 10, Items.DIRT
));
```

数据包注册表对象生成

通过构建 DatapackBuiltinEntriesProvider 并提供 RegistrySetBuilder 来生成数据包注册表对象。

```
@SubscribeEvent
public void gatherData(GatherDataEvent event) {
    event.getGenerator().addProvider(
        event.includeServer(),
        output -> new DatapackBuiltinEntriesProvider(output, event.getLookupProvider(),
            new RegistrySetBuilder().add(/* ... */), Set.of(MOD_ID))
    );
}
```

RegistrySetBuilder: - .add(registryKey, bootstrap -> { ... }) —添加注册表条目 - BootstrapContext#register(key, object) —注册对象 - BootstrapContext#lookup(registryKey) —获取 HolderGetter 以引用其他数据包注册表对象或标签

```
new RegistrySetBuilder()
    .add(Registries.CONFIGURED_FEATURE, bootstrap -> {
        bootstrap.register(EXAMPLE_CONFIGURED_FEATURE, new ConfiguredFeature<>(Feature.ORE, /* ... */));
    })
    .add(Registries.PLACED_FEATURE, bootstrap -> {
        HolderGetter<ConfiguredFeature<?, ?>> configured = bootstrap.lookup(Registries.CONFIGURED_FEATURE);
        bootstrap.register(EXAMPLE_PLACED_FEATURE, new PlacedFeature(configured.getOrThrow(EXAMPLE_CONFIGURED_FEATURE), List.of()));
    });
```

配置

Forge 使用基于 TOML 的配置系统，由 NightConfig 库读取。

创建配置

使用 ForgeConfigSpec\$Builder 构建: - #push(name) / #pop() —配置分区 - #build() —构建 ForgeConfigSpec - #configure(constructor) —构建并绑定到持有类

配置值

- #define(path, defaultValue) —基本值
- #defineInRange(path, min, max, type) —范围值
- #defineInList(path, allowedValues, type) —白名单值
- #defineList(path, validator) —列表值
- #defineEnum(path, enumGetter, allowedValues) —枚举值
- 上下文方法: .comment()、.translation()、.worldRestart()

注册配置

在模组构造函数中: ModLoadingContext.get().registerConfig(type, spec, fileName)

配置类型:

| 类型 | 加载端 | 同步客户端 | 默认文件后缀 |
|--------|------|-------|---------|
| CLIENT | 仅客户端 | 否 | -client |
| COMMON | 双端 | 否 | -common |
| SERVER | 仅服务端 | 是 | -server |

配置事件

ModConfigEvent\$Loading / ModConfigEvent\$Reloading (模组事件总线)

按键映射

按键映射 (Key Mapping) 将特定操作绑定到输入 (鼠标点击、按键等)。

注册

在 RegisterKeyMappingsEvent (模组事件总线, 仅物理客户端) 中注册。

创建

```
new KeyMapping(  
    "key.examplemod.example",    // 翻译键 (显示名称)  
    KeyConflictContext.IN_GAME,  // 冲突上下文  
    KeyModifier.SHIFT,           // 修饰键 (可选)  
    InputConstants.Type.KEYSYM,  // 输入类型  
    GLFW.GLFW_KEY_P,            // 默认键  
    "key.categories.misc"       // 分类  
)
```

冲突上下文

- UNIVERSAL —所有上下文
- GUI —仅在屏幕打开时
- IN_GAME —仅在屏幕未打开时

按键修饰符

KeyModifier.CONTROL / SHIFT / ALT / NONE

检查按键

- **游戏中:** 在 ClientTickEvent (Forge 事件总线) 中使用 #consumeClick()
- **GUI 中:** 在 #keyPressed / #mouseClicked 中使用 #isActiveAndMatches

游戏测试

Game Test 是游戏内的单元测试系统，可并行运行大量测试。

创建测试

1. 创建结构模板 (.nbt 文件)，放置在 data/<namespace>/structures
2. 编写测试方法: @GameTest + Consumer<GameTestHelper>
3. 注册测试: @GameTestHolder(modId) 或 RegisterGameTestsEvent

关键方法

- #succeed / #succeedIf / #succeedWhen / #succeedOnTickWhen —成功标记
- #runAtTickTime / #runAfterDelay / #onEachTick —调度操作
- #absolutePos / #relativePos —坐标转换

批处理

@BeforeBatch(setup) / @AfterBatch(teardown) / @GameTest(batch="name")

运行测试

/test run <name> / runall / runthis / runthese / runfailed

构建配置

- forge.enabledGameTestNamespaces —启用的命名空间
- gradlew runGameTestServer —自动运行并返回退出码

Forge 更新检查器

Forge 提供轻量的可选更新检查框架。如果有可用更新，主菜单的 'Mods' 按钮和模组列表会显示闪烁图标。**不会**自动下载更新。

配置

在 mods.toml 中设置 updateJSONURL 指向更新 JSON 文件。

更新 JSON 格式

```
{
  "homepage": "< 下载页面 >",
  "<mcversion>": {
    "<modversion>": "< 更新日志 >"
  },
  "promos": {
    "<mcversion>-latest": "<modversion>",
    "<mcversion>-recommended": "<modversion>"
  }
}
```

获取检查结果

VersionChecker#getResult(IModInfo): - FAILED —无法连接 - UP_TO_DATE —当前版本等于推荐版本 - OUTDATED —有新推荐或最新版本 - BETA_OUTDATED —有新的最新版本 - PENDING —检查尚未完成

调试分析器

Minecraft 提供调试分析器来查找耗时代码。按 F3 + L 启动，10 秒后自动停止（或再次按 F3 + L 提前停止）。

结果保存在 debug/profiling/< 日期 >-< 世界名 >-< 版本号 >.zip 中。

读取结果

profiling.txt 中每行格式：[深度] 名称 - 父级耗时% / 总耗时%

```
[00] levels - 96.70%/96.70%
[01] |   Level Name - 99.76%/96.47%
[02] |   |   tick - 99.31%/95.81%
[03] |   |   |   entities - 47.72%/45.72%
```

分析自定义代码

```
ProfilerFiller#push("sectionName");
// 要分析的代码
ProfilerFiller#pop();
```

可从 Level、MinecraftServer 或 Minecraft 实例获取 ProfilerFiller。

访问转换器

Access Transformer（简称 AT）用于扩大类、方法和字段的可见性以及修改 final 标志。允许模组访问和修改其他类中原本不可访问的成员。

完整规范见 Minecraft Forge GitHub。

添加 AT

在 build.gradle 中添加一行：

```
minecraft {
  accessTransformer = file('src/main/resources/META-INF/accesstransformer.cfg')
}
```

添加或修改后需刷新 Gradle 项目。生产环境中 Forge 仅搜索 META-INF/accesstransformer.cfg。

语法

注释 (# 开头)

<access modifier> <fully qualified class>

<access modifier> <fully qualified class> <field name>

<access modifier> <fully qualified class> <method name>(<params>)<return>

- 访问修饰符: public > protected > default > private
- +f / -f: 添加/移除 final 标志
- 字段和方法使用 **SRG 名称**
- 内部类用 \$ 分隔

类型描述符

B=byte, C=char, D=double, F=float, I=int, J=long, S=short, Z=boolean, [= 数组, L< 类 >;= 引用类型, V=void

示例

public 化 Crypt 中的 ByteArrayToKeyFunction 接口

public net.minecraft.util.Crypt\$ByteArrayToKeyFunction

protected 化并移除 MinecraftServer 中 random 的 final

protected-f net.minecraft.server.MinecraftServer f_129758_

public 化 Util 中的 makeExecutor 方法

public net.minecraft.Util m_137477_(Ljava/lang/String;)Ljava/util/concurrent/ExecutorService;

!!! warning AT 只修改直接引用的方法, 重写方法不会被转换。安全的转换目标: private 方法、final 方法 (或 final 类中的方法)、static 方法。

开始 Forge 开发

如果你决定为 Forge 本身做贡献, 需要一些特殊步骤来开始。

Fork 和克隆

1. Fork MinecraftForge 仓库
2. 克隆到本地: `git clone https://github.com/<User>/MinecraftForge`
3. 为每个 PR 创建独立分支

设置开发环境

- Eclipse: `./gradlew setup` → `./gradlew genEclipseRuns` → 导入 Gradle 项目
- IntelliJ IDEA: 导入项目 → 运行 setup 任务 → 运行 genIntelliJRuns 任务

修改和提交 PR

- 修改代码必须在 “Forge” 子项目中进行, 不要在 “Clean” 项目中修改
- 修改 Minecraft 代码后运行 `genPatches` 生成补丁
- 通过 GitHub 提交 Pull Request

Pull Request 指南

Forge 代码分为两类:

补丁 (Patches)

- 直接修改 Minecraft 源码, 应**尽可能小**
- 策略: 插入单行触发事件/钩子, 主体代码放在补丁外

- Review 时会关注补丁大小

Forge 代码

- 普通的 Java 代码（事件、兼容性功能等），无大小限制
- Review 时会关注代码整洁度和文档

准则

1. **解释必要性** —说明为什么需要这个改动，提供具体用例
2. **证明能工作** —添加示例模组或 JUnit 测试
3. **避免破坏性变更** —不破坏旧版本的二进制兼容性。例外：新 Minecraft 版本初期、紧急修复
4. **保持耐心** —Code review 不是针对个人

旧版本文档

Forge 使用 LTS 系统，只有最新版本和当前 LTS 版本有易访问的文档。旧版本文档仅供参考。

历史文档版本

| 版本 | 准确率 | 链接 |
|--------|------|---|
| 1.12.x | 100% | https://docs.minecraftforge.net/en/1.12.x/ |
| 1.16.x | 85% | https://docs.minecraftforge.net/en/1.16.x/ |
| 1.18.x | 90% | https://docs.minecraftforge.net/en/1.18.x/ |
| 1.19.2 | 90% | https://docs.minecraftforge.net/en/1.19.2/ |
| 1.19.x | 90% | https://docs.minecraftforge.net/en/1.19.x/ |

!!! important 旧版本文档仅供参考。不要在 Forge Discord 或论坛上寻求旧版本帮助，**你不会得到支持**。

RetroGradle

一项归档计划，将旧版 ForgeGradle 1.x 工具链升级到现代 ForgeGradle 4.x+，以保留所有历史版本的 Minecraft Forge。仅用于**保存**，不用于开发旧版本模组。

移植到 Minecraft 1.20

以下是从旧版本移植到当前版本的指南：

| 从 → 到 | 指南 |
|------------------|---------------------------|
| 1.12 → 1.13/1.14 | Primer by williewillus |
| 1.14 → 1.15 | Primer by williewillus |
| 1.15 → 1.16 | Primer by 50ap5ud5 |
| 1.16 → 1.17 | Primer by 50ap5ud5 |
| 1.19.2 → 1.19.3 | Primer by ChampionAsh5357 |
| 1.19.3 → 1.19.4 | Primer by ChampionAsh5357 |
| 1.19.4 → 1.20 | Primer by ChampionAsh5357 |